

Базы данных

Лабораторная работа 1

Реляционная модель и ключи

Студент: Айтжанов Ердаулет

Дата: 15 сентября 2026

Содержание

Часть 1. Ключи

Задание 1.1. Суперключи и потенциальные ключи

Задание 1.2. Внешние ключи

Часть 2. ER-диаграммы

Задание 2.1. Больница

Задание 2.2. Интернет-магазин

Часть 4. Нормализация

Задание 4.1. Нормализация StudentProject до 3НФ

Задание 4.2. Нормализация CourseSchedule до НФБК

Часть 5. Проектирование

Задание 5.1. Студенческие клубы и организации

В методичке после части 2 сразу идёт часть 4, так что части 3 здесь тоже нет. Первичные ключи подчёркнуты во всех схемах, внешние ключи перечислены под той таблицей, к которой относятся. Три ER-диаграммы выполнены в нотации Чена, каждая на отдельной странице, повернутой поперёк, чтобы поместилась.

Часть 1. Ключи

Задание 1.1. Суперключи и потенциальные ключи

Employee(EmpID INT, SSN CHAR(11), Email VARCHAR(254), Phone VARCHAR(20), Name VARCHAR(50), Department VARCHAR(30), Salary DECIMAL(10,2))

Атрибут	Тип SQL	Роль
EmpID	INT NOT NULL IDENTITY	Первичный ключ (СК1). Суррогатный, выдаётся отделом кадров, повторно не используется
SSN	CHAR(11)	СК2, UNIQUE. Фиксированная ширина, NNN-NN-NNNN вместе с дефисами
Email	VARCHAR(254)	СК3, UNIQUE. 254 — предел длины адреса почтового ящика по RFC 5321
Phone	VARCHAR(20)	Не уникален. Хватает места для +7, разделителей и добавочных номеров
Name	VARCHAR(50)	NOT NULL, не уникален
Department	VARCHAR(30)	Не уникален
Salary	DECIMAL(10,2)	CHECK (Salary > 0). Точный денежный тип, никакого FLOAT

Внешних ключей в этом отношении, как оно задано, нет.

Я предполагаю, что отдел кадров выдаёт каждому сотруднику один EmpID и никогда его не меняет, что SSN принадлежит ровно одному человеку и что два сотрудника не делят один почтовый ящик. О Phone не сказано вообще ничего. SSN и Email допускают NULL, потому что при приёме на работу значение действительно может быть неизвестно, так что их ограничения UNIQUE срабатывают только на строках, где значение есть.

Суперключи

Любое надмножество потенциального ключа — суперключ:

1. {EmpID}
2. {SSN}
3. {Email}
4. {EmpID, Name}
5. {Email, Salary}
6. {SSN, Phone, Department}
7. {EmpID, SSN, Email, Phone, Name, Department, Salary}

Первые три — ещё и потенциальные ключи. Номера с 4 по 7 — нет: каждый из них строго содержит суперключ поменьше и потому не минимален. {EmpID, Name} прекрасно идентифицирует строку, просто тащит с собой балласт.

Потенциальные ключи

{EmpID}, {SSN}, {Email}.

Каждый из них — суперключ по правилам выше. Каждый минимален, потому что состоит из одного атрибута, а единственное собственное подмножество одноэлементного множества — пустое множество, которое служит ключом только в отношении не более чем из одной строки.

Больше ничего не подходит. Потенциальный ключ не может строго содержать другой потенциальный ключ, поэтому ни одно множество с EmpID, SSN или Email в список не попадает. Остаются подмножества {Phone, Name, Department, Salary}, а два сотрудника могут совпадать по всем четырём сразу (один общий добавочный номер, одно имя, один отдел, одна зарплата), так что и никакое их подмножество не годится. Department отпадает уже по приведённым данным: IT встречается в двух строках.

Первичный ключ

Я взял EmpID. Он не несёт бизнес-смысла, поэтому никакое событие в реальном мире не заставит его измениться, а при размере в 4 байта это самый дешёвый внешний ключ, который придётся хранить Payroll, Attendance и остальным таблицам.

У каждого из двух других есть свой изъян. Email нестабилен: смена фамилии или ребрендинг домена после слияния его переписуют. SSN у человека, который ждёт рабочую визу, на момент приёма может ещё не быть, а столбец, допускающий NULL, первичным ключом быть не может. К тому же SSN — регулируемые персональные данные, а первичный ключ располагается по всем дочерним таблицам, индексам и файлам логов, и это последнее место, где должен лежать национальный идентификатор. Оба остаются в схеме как альтернативные ключи.

```
CREATE TABLE Employee (  
    EmpID      INT          NOT NULL IDENTITY(1,1),  
    SSN        CHAR(11)     NULL,  
    Email      VARCHAR(254) NULL,  
    Phone      VARCHAR(20)  NULL,  
    Name       VARCHAR(50)  NOT NULL,  
    Department VARCHAR(30)  NOT NULL,  
    Salary     DECIMAL(10,2) NOT NULL CHECK (Salary > 0),  
    CONSTRAINT PK_Employee PRIMARY KEY (EmpID),  
    CONSTRAINT UQ_Emp_SSN   UNIQUE (SSN),  
    CONSTRAINT UQ_Emp_Email UNIQUE (Email)  
);
```

Могут ли два сотрудника иметь один номер телефона?

Да. В трёх приведённых строках случайно оказались три разных номера (555-0101, 555-0102, 555-0103), но о схеме это не говорит ничего.

Логика здесь работает только в одну сторону. Дубликат в данных сразу опровергает утверждение о ключе: одного контрпримера хватает, чтобы разбить утверждение, которое претендует на то, что выполняется всегда, и именно так выше был отброшен Department. Обратное неверно: отсутствие дубликатов ничего не доказывает. Три строки дают всего три пары для сравнения, а уникальность на трёх строках случайно получается почти на любых данных. Ключи берутся из бизнес-правил о схеме, так что пример данных может опровергнуть ключ, но никогда не может его подтвердить.

Да и общие номера — обычное дело. Два человека в одном кабинете отвечают на один добавочный, весь отдел кадров записывает номер ресепшена, а корпоративный мобильный передают новичку, пока строка уволившегося ещё открыта. Кроме того, Phone по своей природе многозначный атрибут (рабочий, мобильный, экстренный контакт), и это отдельная причина не строить на нём ключ.

Я назвал бы Phone потенциальным ключом, только если бы предприятие сформулировало правило вроде «каждому сотруднику выдаётся одна корпоративная линия, и линии не делятся», а мы записали бы его в схему как Phone VARCHAR(20) NOT NULL UNIQUE. Но и тогда это был бы альтернативный ключ, а не первичный, потому что номера меняются, когда люди пересаживаются.

Отношение B: Registration (регистрация на курсы)

Registration(StudentID INT, CourseCode VARCHAR(10), Section CHAR(2), Semester VARCHAR(6), Year SMALLINT, Grade VARCHAR(2), Credits DECIMAL(3,1))

Атрибут	Тип SQL	Роль
<u>StudentID</u>	INT NOT NULL	Часть первичного ключа, внешний ключ на Student
<u>CourseCode</u>	VARCHAR(10) NOT NULL	Часть первичного ключа, часть внешнего ключа. Например, CS201
<u>Section</u>	CHAR(2) NOT NULL	Часть первичного ключа, часть внешнего ключа. Метки уникальны только внутри курса
<u>Semester</u>	VARCHAR(6) NOT NULL	Часть первичного ключа, часть внешнего ключа. CHECK (Semester IN ('Fall', 'Spring', 'Summer'))
<u>Year</u>	SMALLINT NOT NULL	Часть первичного ключа, часть внешнего ключа. CHECK (Year BETWEEN 1990 AND 2100)
Grade	VARCHAR(2) NULL	Неключевой. NULL означает, что курс ещё идёт, а A и B+ разной длины
Credits	DECIMAL(3,1) NOT NULL	Неключевой. Десятичный, чтобы поместилась лабораторная на 1.5 кредита

Внешние ключи:

- StudentID -> Student(StudentID)
- (CourseCode, Section, Semester, Year) -> CourseOffering(CourseCode, Section, Semester, Year)

Наименьший первичный ключ

Пять атрибутов: (StudentID, CourseCode, Section, Semester, Year).

BR2 говорит, что студент не может зарегистрироваться на одну и ту же секцию курса в одном и том же семестре, и эта фраза и есть утверждение, что пятёрка уникальна. Grade и Credits ею определяются, поэтому в неё не входят.

Я читаю BR2 буквально: оно запрещает только повтор (студент, курс, секция, семестр), а сидеть в секции A и секции B одного курса в один семестр студент может. Поэтому Section входит в ключ.

Почему нужен каждый из пяти

Уберите любой из атрибутов, и я смогу написать две строки, которые законны по заданным правилам, но совпадают по всему оставшемуся.

Без StudentID запись (CS201, A, Fall, 2024) подходит каждому студенту секции, а в секции много студентов. Без CourseCode метки секций сталкиваются, потому что «секция A» есть в каждом курсе, и студент 1001 в CS201-A и тот же студент в MATH101-A превращаются в одну строку. Без Section записи этого студента на CS201-A и CS201-B схлопываются в одну. Без Semester BR1 позволяет 1001 пройти CS201-A заново в Spring 2024 и ещё раз в Fall 2024. Без Year не отличить Fall 2024 от Fall 2025, потому что названия семестров повторяются каждый год.

Ни одно собственное подмножество не является суперключом, поэтому пять атрибутов образуют потенциальный ключ, а не просто суперключ.

Есть ли ещё потенциальные ключи?

Нет. Зависимости такие: K -> Grade и (CourseCode, Section, Semester, Year) -> Credits, где K — ключ из пяти атрибутов. StudentID, CourseCode, Section, Semester и Year не встречаются в правой части ни одной функциональной зависимости (ФЗ), так что ничто их не выводит, и каждый потенциальный ключ обязан содержать все пять. А эти пять уже определяют всё, поэтому K — единственный потенциальный ключ, а Grade и Credits остаются неключевыми.

Стоит отметить для части 4

BR3 даёт (CourseCode, Section, Semester, Year) -> Credits. Детерминант — собственное подмножество первичного ключа, а Credits — неключевой атрибут, так что это частичная зависимость, и Registration находится в первой нормальной форме (1НФ, 1NF), но не во второй (2НФ, 2NF). Число кредитов секции повторяется по разу на каждого записанного студента. Вынос Credits в CourseOffering это исправляет.

Задание 1.2. Внешние ключи

Первичные ключи подчёркнуты, внешние ключи выделены курсивом.

Отношение	Схема (атрибут : тип SQL)
Student	Student(<u>StudentID</u> : INT, Name : VARCHAR(100) NOT NULL, Email : VARCHAR(255) NOT NULL UNIQUE, Major : CHAR(4) NULL, AdvisorID : INT NULL)
Professor	Professor(<u>ProfID</u> : INT, Name : VARCHAR(100) NOT NULL, DeptCode : CHAR(4) NOT NULL, Salary : DECIMAL(10,2) NULL)
Course	Course(<u>CourseID</u> : CHAR(8), Title : VARCHAR(150) NOT NULL, Credits : SMALLINT NOT NULL, DepartmentCode : CHAR(4) NOT NULL)
Department	Department(<u>DeptCode</u> : CHAR(4), DeptName : VARCHAR(100) NOT NULL UNIQUE, Budget : DECIMAL(12,2) NOT NULL, ChairID : INT NULL UNIQUE)
Enrollment	Enrollment(<u>StudentID</u> : INT, <u>CourseID</u> : CHAR(8), <u>Semester</u> : CHAR(6), Grade : VARCHAR(2) NULL)

Professor.DeptCode — это атрибут, который в задании назван Professor.Department. Я переименовал его, потому что он хранит код кафедры, а не кафедру, и потому что один и тот же домен встречался под тремя разными именами (Professor.Department, Course.DepartmentCode, Department.DeptCode). В Enrollment все три атрибута ключа — ещё и внешние ключи, поэтому они одновременно подчёркнуты и выделены курсивом.

Внешние ключи

Шесть внешних ключей вытекают из таблиц в том виде, в каком они заданы. Ещё двух нет, хотя они нужны; я добавил их как FK7 и FK8.

#	Дочерняя таблица	Столбец (столбцы)	Ссылается на	Кардинальность	NULL / участие	ON DELETE / ON UPDATE
FK1	Student	AdvisorID (INT)	Professor.ProfID	N:1, один преподаватель курирует многих студентов	допускает NULL, участие с обеих сторон частичное	SET NULL / NO ACTION
FK2	Professor	DeptCode (CHAR(4))	Department.DeptCode	N:1, на кафедре работает много преподавателей	NOT NULL, со стороны Professor участие полное	RESTRICT / CASCADE
FK3	Course	DepartmentCode (CHAR(4))	Department.DeptCode	N:1, кафедре принадлежит много курсов	NOT NULL, со стороны Course участие полное	RESTRICT / CASCADE
FK4	Department	ChairID (INT)	Professor.ProfID	1:1, но только потому, что заодно объявлен UNIQUE(ChairID)	допускает NULL, участие с обеих сторон частичное	SET NULL / NO ACTION
FK5	Enrollment	StudentID, часть первичного ключа	Student.StudentID	N:1, у студента много записей на курсы	NOT NULL, со стороны Enrollment участие полное	CASCADE / NO ACTION
FK6	Enrollment	CourseID, часть первичного ключа	Course.CourseID	N:1, на курс приходится много записей	NOT NULL, со стороны Enrollment участие полное	RESTRICT / CASCADE
FK7	Student	Major (CHAR(4))	Department.DeptCode	N:1	допускает NULL, специальность не выбрана	RESTRICT / CASCADE
FK8	Enrollment	Semester (CHAR(6)), часть первичного ключа	Semester.SemesterCode	N:1	NOT NULL	RESTRICT / CASCADE

FK7 и FK8 — рекомендации. В заданной схеме их нет.

Ссылочные действия вытекают из двух вопросов: может ли меняться родительский ключ и может ли дочерняя строка пережить родительскую. ProfID и StudentID — суррогатные ключи, которые никогда не меняются, поэтому всё, что на них ссылается, использует NO ACTION при обновлении. DeptCode и CourseID — естественные ключи, и их действительно перекодируют при реорганизации факультета или перенумерации каталога, поэтому для них выбран каскад. При удалении преподаватель, который увольняется, не должен забирать с собой своих подопечных студентов, а уходящий заведующий не должен забирать с собой кафедру, поэтому FK1 и FK4 обнуляют дочерний столбец. Оба столбца допускают NULL, так что это допустимо. FK2, FK3 и FK6 запрещают удаление, потому что преподавателей нужно перевести, а курсы переприкрепить, прежде чем кафедра исчезнет, а запись на курс с выставленной оценкой должна пережить исключение курса из каталога. FK5 — единственный каскад, так как строка записи на курс без студента лишена смысла. Асимметрия между FK5 и FK6 сделана намеренно: существование Enrollment зависит от студента, а на курс только ссылается.

В FK4 есть одна деталь, которую легко упустить. Внешний ключ сам по себе всегда даёт N:1, поэтому один только ChairID позволил бы двум кафедрам называть заведующим одного и того же преподавателя. Связь становится 1:1 именно благодаря отдельному UNIQUE(ChairID).

Enrollment как таблица-связка

Student и Course связаны как M:N, потому что студент проходит много курсов, а курс проходят многие студенты. Внешний ключ однозначный, поэтому ни одна из таблиц не может хранить ключ другой, и связь нельзя сохранить напрямую. Enrollment — таблица-связка, которая её реализует.

Её первичный ключ — (StudentID, CourseID, Semester). Semester должен входить в ключ, потому что студент может повторно пройти тот же курс в одном из следующих семестров, а один только (StudentID, CourseID) это запретил бы. Все три атрибута входят в первичный ключ, поэтому они неявно NOT NULL, а поскольку два из них ещё и внешние ключи, Enrollment участвует в обеих связях полностью. Сам по себе внешний ключ не требует NOT NULL: Student.AdvisorID и Department.ChairID — оба внешние ключи, допускающие NULL. В нотации Чена это делает Enrollment слабой сущностью, соединённой со Student и Course идентифицирующими связями, где Semester — дискриминатор, а Grade — описательный атрибут связи, а не какого-либо из участников. (StudentID, CourseID, Semester, Grade) — суперключ, но не потенциальный ключ, потому что он не минимален: Grade функционально определяется тремя остальными, так что отбросить его ничего не стоит.

Цикл между Professor и Department

Два внешних ключа указывают друг на друга. Professor.DeptCode ссылается на Department.DeptCode, а Department.ChairID ссылается на Professor.ProfID. Это ссылочный цикл из двух узлов, и если оба столбца NOT NULL при обычной немедленной проверке ограничений, базу данных вообще нельзя заполнить. Чтобы вставить преподавателя, его кафедра уже должна существовать. Чтобы вставить кафедру, уже должен существовать её заведующий.

Ни одна из таблиц не может принять свою первую строку. Схема корректна, но ни одно непустое состояние недостижимо. Заметить это трудно потому, что в примере данных оно никогда не проявляется: пример данных попросту нельзя построить.

Я исправил это, объявив `Department.ChairID` как `NULL UNIQUE` и оставив `Professor.DeptCode` `NOT NULL`. Тогда строки вставляются так: сначала кафедра с `NULL` вместо заведующего, потом преподаватель, потом `UPDATE`, который задаёт `ChairID`, а после этого студенты, курсы и, наконец, записи на курсы. Цена в том, что теперь можно представить кафедру без заведующего. Если бизнес-правило действительно звучит как «у каждой кафедры есть заведующий», его придётся вынести из `NOT NULL` в триггер или в ночную проверку.

```
CREATE TABLE Department (
  DeptCode CHAR(4) NOT NULL,
  DeptName VARCHAR(100) NOT NULL,
  Budget DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  ChairID INT NULL, -- именно NULL разрывает цикл
  CONSTRAINT PK_Department PRIMARY KEY (DeptCode),
  CONSTRAINT UQ_Department_Name UNIQUE (DeptName),
  CONSTRAINT UQ_Department_Chair UNIQUE (ChairID) -- делает FK4 связью 1:1
);

-- Professor создаётся следующей, а цикл замыкается уже потом:
ALTER TABLE Department
  ADD CONSTRAINT FK_Department_Chair FOREIGN KEY (ChairID)
    REFERENCES Professor (ProfID)
    ON DELETE SET NULL ON UPDATE NO ACTION;
```

Есть ещё два способа. Оба столбца могут остаться `NOT NULL`, если внешний ключ на заведующего объявить `DEFERRABLE INITIALLY DEFERRED` и вставлять обе строки в одной транзакции; это поддерживают PostgreSQL и Oracle, MySQL игнорирует, а в SQL Server этого нет вовсе. Или заведующего можно вынести в отдельную таблицу `DepartmentChair`, что убирает цикл, потому что оба её внешних ключа тогда указывают наружу. Я оставил столбец, допускающий `NULL`, поскольку это наименьшее изменение для пяти таблиц, заданных в работе, но именно к отдельной таблице я перешёл бы, как только историю заведующих понадобится хранить для аудита.

Замечания по проектированию

- У `Student.Major` нет внешнего ключа, так что это свободный текст, и 'CS', 'Comp Sci' и 'Basket Weaving' одинаково допустимы. Минимальное исправление — FK7. Лучше завести таблицу `Major`, поскольку одна кафедра обычно предлагает несколько программ, а двойные специальности потребовали бы ещё и таблицы-связки `StudentMajor`.
- У `Semester` нет контролируемого словаря и нет родительской таблицы, на которую можно сослаться. '2024FA', 'F24' и 'Fall2024' могут одновременно оказаться в первичном ключе, незаметно разбивая один семестр на три и ломая правило повторного прохождения. Нужен справочник `Semester` или хотя бы `CHECK` на формат кода.
- У `Grade` нет ограничения домена, а его `NULL` перегружен: курс ещё идёт, студент отказался от курса или оценка ещё не подана. `CHECK` по списку допустимых оценок решает первую часть, а справочник `GradeScale` с баллами позволил бы считать GPA одним соединением.
- Ничто не фиксирует, кто что преподаёт, и ничто не мешает записаться на пару (`CourseID`, `Semester`), в которой этот курс в том семестре вовсе не предлагался. Два независимых внешних ключа не могут ограничить комбинацию. Здесь нужна сущность `Offering` с ключом из этой пары плюс секции, а `Enrollment` надо перенацелить на неё; это заменило бы ключ из трёх атрибутов, описанный выше.
- `Email` — второй потенциальный ключ, который так и не был объявлен `UNIQUE`.
- На числа нет ограничений по значениям, поэтому отрицательная зарплата или бюджет и курс с нулём кредитов сейчас вполне допустимы.
- «Заведующий должен работать на кафедре, которой заведует» нельзя записать простым внешним ключом, потому что это правило связывает два атрибута из двух таблиц. Его можно реализовать составным внешним ключом (`ChairID`, `DeptCode`), ссылающимся на объявленный `UNIQUE(ProfID, DeptCode)` в `Professor`; это допустимый, хотя и неминимальный суперключ. Это ограничение должно использовать `NO ACTION` при обновлении, а не `CASCADE`, иначе заведующий, переходящий на другую кафедру, попытался бы переписать первичный ключ самой кафедры.

Часть 2. ER-диаграммы

Задание 2.1. Больница

Из сценария вышло восемь типов сущностей: семь сильных и одна слабая.

Сущность	Тип	Первичный ключ
Patient	сильная	<u>PatientID</u>
Doctor	сильная	<u>DoctorID</u>
Department	сильная	<u>DeptCode</u>
Room	слабая	(<u>DeptCode</u> , <u>RoomNumber</u>)
Appointment	сильная (ассоциативная)	<u>AppointmentID</u>
Prescription	сильная (ассоциативная)	<u>PrescriptionID</u>
Medication	сильная	<u>MedicationID</u>
InsurancePolicy	сильная	<u>PolicyID</u>

Patient, Doctor и Department — сильные: задание даёт каждой из них собственный идентификатор, и каждая существует сама по себе, независимо от всего остального.

Appointment и Prescription начинались как связи, и я поднял их до сущностей. Ключ обычной связи M:N между Patient и Doctor — это (PatientID, DoctorID), а он разрешает ровно один приём на пару «пациент — врач» за всё время, тогда как пациенты возвращаются к тому же врачу. У Prescription та же беда, и вдобавок это документ со своим жизненным циклом: дата выписки, повторные выдачи, статус. На обе должны ссылаться другие элементы модели, а в нотации Чена концом связи может быть только сущность.

Medication — отдельная сущность, а не текстовое поле в Prescription. Иначе одно и то же лекарство в разных строках напишут тремя разными способами, новый препарат из формуляра нельзя будет внести, пока его кто-нибудь не выпишет, а удаление последнего рецепта на редкое лекарство удалит и само лекарство.

InsurancePolicy вынесена отдельно, а не сделана составным атрибутом Patient: у пациента могут быть одновременно основная и дополнительная страховки, а один семейный полис покрывает нескольких пациентов под одним номером. Альтернативный ключ — (ProviderName, PolicyNumber).

Почему Room — слабая сущность

В задании сказано, что палаты нумеруются внутри отделений и что палата 101 в кардиологии — не та же палата, что 101 в неврологии. Одна эта фраза даёт все три условия.

RoomNumber не уникален по всему множеству сущностей, и контрпример даёт сама эта фраза. RoomType, BedCapacity и Floor тоже не уникальны, так что своего ключа у Room нет. Её идентичность достраивает отделение, которое содержит палату: «нумеруются внутри отделений» означает, что RoomNumber — частичный ключ, уникальный только внутри одного владельца. Связь Department с Room — идентифицирующая связь Belongs_To, двойной ромб, 1:N, с двойной линией со стороны Room, потому что палата не может существовать без отделения.

Итого PK(Room) = (DeptCode, RoomNumber): ключ владельца плюс частичный ключ.

Суррогатный RoomID я рассматривал, он сделал бы Room сильной. Вариант рабочий, но тогда ничто не мешает двум строкам разом объявить себя палатой 101 в кардиологии, а ведь именно это ограничение в задании и записано.

Атрибуты

Частичный ключ Room подчёркнут на диаграмме пунктиром.

Сущность	Простые	Составные	Многозначные	Производные
Patient	<u>PatientID</u> , BirthDate, Gender, RegistrationDate	Name (First, Middle, Last), Address (Street, City, State, Zip)	PhoneNumbers	Age
Doctor	<u>DoctorID</u> , HireDate, LicenseNumber	Name (First, Middle, Last), OfficeLocation (Building, Floor, OfficeNumber)	Specializations, PhoneNumbers	YearsOfExperience
Department	<u>DeptCode</u> , DeptName, PhoneExtension	Location (Building, Floor)	нет	NumberOfDoctors
Room	<u>RoomNumber</u> (частичный ключ), RoomType, BedCapacity, Floor	нет	нет	нет
Appointment	<u>AppointmentID</u> , AppointmentDateTime, DurationMinutes, Purpose, Notes, Status	нет	нет	EndTime
Prescription	<u>PrescriptionID</u> , IssueDate, Dosage, Frequency, Instructions, DurationDays, RefillsAllowed	нет	нет	EndDate
Medication	<u>MedicationID</u> , MedName, Form, Strength, Manufacturer, UnitPrice, RequiresPrescription	нет	нет	нет
InsurancePolicy	<u>PolicyID</u> , ProviderName, PolicyNumber, GroupNumber, CoverageType, ValidFrom, ValidTo, CoveragePercent	нет	нет	IsActive

Собственные атрибуты есть у трёх связей: AssignedSince у Works_In, AppointedOn у Managed_By и CoverageRole, RelationToSubscriber, EffectiveFrom, EffectiveTo у Covered_By.

Каждый составной атрибут — родительский овал, от которого отходят его составляющие. Address разбит ровно так, как он перечислен в задании. OfficeLocation у Doctor остаётся составным атрибутом, а не внешним ключом на Room, потому что кабинет для приёма — административное помещение, а не клиническая палата внутри отделения.

Многозначные атрибуты — двойные овалы: телефоны пациента, телефоны врача и специализации врача, все они стоят в задании во множественном числе. Если телефон врача имелся в виду как один рабочий номер, он превращается в простой атрибут OfficePhone, а таблица DoctorPhone ниже исчезает; больше ничего не меняется.

Производные атрибуты — пунктирные овалы, и ни один из них не хранится. Age считается из BirthDate и сегодняшней даты, поэтому сохранённая копия устареет сама собой, и YearsOfExperience работает так же от HireDate. NumberOfDoctors — это COUNT по Works_In. EndTime — это AppointmentDateTime плюс DurationMinutes, EndDate — IssueDate плюс DurationDays, а IsActive истинно, когда сегодняшний день попадает между ValidFrom и ValidTo.

Связи

№	Связь	Между	Кардинальность	Участие	Собственные атрибуты
R1	Belongs_To (идентифицирующая)	Department, Room	1:N	Department частичное, Room полное	нет
R2	Works_In	Doctor, Department	N:1	Doctor полное, Department полное	AssignedSince
R3	Managed_By	Department, Doctor (роль: head)	1:1	Department полное, Doctor частичное	AppointedOn
R4	Books	Patient, Appointment	1:N	Patient частичное, Appointment полное	нет
R5	Conducts	Doctor, Appointment	1:N	Doctor частичное, Appointment полное	нет
R6	Held_In	Appointment, Room	N:1	Appointment частичное, Room частичное	нет
R7	Prescribed_By	Doctor, Prescription	1:N	Doctor частичное, Prescription полное	нет
R8	Prescribed_For	Patient, Prescription	1:N	Patient частичное, Prescription полное	нет
R9	Of_Medication	Medication, Prescription	1:N	Medication частичное, Prescription полное	нет
R10	Issued_During	Appointment, Prescription	1:N	Appointment частичное, Prescription частичное	нет
R11	Covered_By	Patient, InsurancePolicy	M:N	Patient частичное, InsurancePolicy полное	CoverageRole, RelationToSubscriber, EffectiveFrom, EffectiveTo

Связь M:N «пациент посещает врача», которую требует задание, — это R4 в композиции с R5, а её описательные данные (дата и время, цель, заметки, статус) лежат на Appointment.

Works_In и Managed_By соединяют одни и те же два типа сущностей, но говорят о разном: одна про штат, другая про руководство. Поэтому ромбов два, и поэтому линия от Doctor к Managed_By несёт метку роли «head». В Works_In участие полное с обеих сторон: врач состоит в штате ровно одного отделения, а отделение без штата — не отделение. В Managed_By участие полное со стороны Department и частичное со стороны Doctor, потому что почти никто из врачей ничем не заведует, и для 1:1 это обычная картина. Двойная линия говорит только то, что минимум равен 1; правило вроде «не меньше трёх врачей на отделение» потребовало бы (3,N) или CHECK.

В Held_In участие частичное с обеих сторон. У приёма может не быть палаты (телефонные консультации, ещё не распределённые слоты), а в палате может не проходить ни одного приёма. Внешний ключ здесь составной, потому что составной ключ у Room.

R7, R8 и R9 вместе реализуют тернарный факт Doctor x Patient x Medication, описанный в задании. Такое разбиение ничего не теряет: каждый экземпляр Prescription несёт ровно одно значение на каждой ветви, поэтому тройка восстанавливается соединением по PrescriptionID.

Covered_By — M:N в обе стороны: у одного пациента может быть основная плюс дополнительная страховка, а один семейный полис покрывает и держателя, и иждивенцев. Участие частичное со стороны Patient, потому что бывают пациенты, платящие сами, и полное со стороны InsurancePolicy, потому что полис вносится только тогда, когда его кто-то держит. Сделать связь N:1, а полис — справочником, значило бы запретить двойное покрытие и дублировать данные страховщика для каждого иждивенца.

Ключи

Сущность	Первичный ключ	Вид	Альтернативные ключи
Patient	<u>PatientID</u>	суррогатный	гарантированных нет
Doctor	<u>DoctorID</u>	суррогатный	{LicenseNumber}
Department	<u>DeptCode</u>	естественный	{DeptName}
Room	<u>(DeptCode, RoomNumber)</u>	ключ владельца + частичный ключ	нет
Appointment	<u>AppointmentID</u>	суррогатный	{DoctorID, AppointmentDateTime}, {PatientID, AppointmentDateTime}
Prescription	<u>PrescriptionID</u>	суррогатный	нет
Medication	<u>MedicationID</u>	суррогатный	{MedName, Strength, Form}
InsurancePolicy	<u>PolicyID</u>	суррогатный	{ProviderName, PolicyNumber}

Два альтернативных ключа Appointment берутся из правил: один врач не может начать две консультации в один и тот же момент, и один пациент тоже. Тройка {PatientID, DoctorID, AppointmentDateTime} — суперключ, но не потенциальный ключ: она содержит обе пары, то есть сократима, и объявить её UNIQUE значило бы разрешить ровно те двойные записи, которые эти две пары запрещают.

У Prescription альтернативного ключа нет. Больница разрешает одному и тому же врачу выписать тому же пациенту второй рецепт на то же лекарство в тот же день, например с исправленной дозировкой, так что даже {DoctorID, PatientID, MedicationID, IssueDate} не суперключ.

Ключ я объявлял только там, где дубликаты запрещает бизнес-правило, а не там, где небольшой пример данных выглядит уникальным. Два пациента по фамилии Иванов с одной датой рождения допустимы, а PolicyNumber уникален только внутри одного страховщика — отсюда и пара.

Реляционная схема

Сильные сущности превращаются в таблицы. Room берёт первичным ключом ключ владельца плюс свой частичный ключ, и эти же столбцы владельца работают как внешний ключ. Составные атрибуты разворачиваются до листьев: Address исчезает, а появляются Street, City, State, Zip. Каждый многозначный атрибут получает свою таблицу, потому что 1НФ не допускает повторяющуюся группу в ячейке. Производные атрибуты не хранятся. Каждая связь 1:N становится внешним ключом на стороне N, с NOT NULL, если участие с этой стороны полное, а связь M:N становится таблицей-связкой.

Department(DeptCode CHAR(4), DeptName VARCHAR(60) NOT NULL UNIQUE, Building VARCHAR(30), Floor SMALLINT, PhoneExtension VARCHAR(10), HeadDoctorID INT NULL UNIQUE, AppointedOn DATE NULL) Внешний ключ: HeadDoctorID -> Doctor(DoctorID) ON DELETE SET NULL. HeadDoctorID и AppointedOn оба пришли из R3.

Doctor(DoctorID INT AUTO_INCREMENT, FirstName VARCHAR(40) NOT NULL, MiddleName VARCHAR(40) NULL, LastName VARCHAR(40) NOT NULL, LicenseNumber CHAR(12) NOT NULL UNIQUE, OfficeBuilding VARCHAR(30), OfficeFloor SMALLINT, OfficeNumber VARCHAR(10), HireDate DATE NOT NULL, DeptCode CHAR(4) NOT NULL, AssignedSince DATE) Внешний ключ: DeptCode -> Department(DeptCode) ON DELETE RESTRICT.

DoctorSpecialization(DoctorID INT, Specialization VARCHAR(60)) Внешний ключ: DoctorID -> Doctor(DoctorID) ON DELETE CASCADE. Само значение входит в ключ, поэтому одному врачу нельзя дважды записать «Cardiology».

DoctorPhone(DoctorID INT, PhoneNumber VARCHAR(20)) Внешний ключ: DoctorID -> Doctor(DoctorID) ON DELETE CASCADE.

Patient(PatientID INT AUTO_INCREMENT, FirstName VARCHAR(40) NOT NULL, MiddleName VARCHAR(40) NULL, LastName VARCHAR(40) NOT NULL, BirthDate DATE NOT NULL, Street VARCHAR(80), City VARCHAR(40), State CHAR(2), Zip CHAR(5), Gender CHAR(1) CHECK (Gender IN ('M','F','O')), RegistrationDate DATE NOT NULL DEFAULT CURRENT_DATE) Внешних ключей нет. Age — не столбец.

PatientPhone(PatientID INT, PhoneNumber VARCHAR(20)) Внешний ключ: PatientID -> Patient(PatientID) ON DELETE CASCADE.

InsurancePolicy(PolicyID INT AUTO_INCREMENT, ProviderName VARCHAR(80) NOT NULL, PolicyNumber VARCHAR(30) NOT NULL, GroupNumber VARCHAR(30) NULL, CoverageType VARCHAR(20), ValidFrom DATE NOT NULL, ValidTo DATE CHECK (ValidTo > ValidFrom), CoveragePercent DECIMAL(5,2) CHECK (CoveragePercent BETWEEN 0 AND 100)) UNIQUE (ProviderName, PolicyNumber) — это альтернативный ключ. IsActive производный и не хранится.

PatientPolicy(PatientID INT, PolicyID INT, CoverageRole VARCHAR(10) CHECK (CoverageRole IN ('primary','secondary')), RelationToSubscriber VARCHAR(20), EffectiveFrom DATE NOT NULL, EffectiveTo DATE NULL) Внешние ключи: PatientID -> Patient(PatientID) ON DELETE CASCADE; PolicyID -> InsurancePolicy(PolicyID) ON DELETE RESTRICT.

Room(DeptCode CHAR(4), RoomNumber VARCHAR(6), RoomType VARCHAR(20), BedCapacity SMALLINT, Floor SMALLINT). Внешний ключ: DeptCode ссылается на Department(DeptCode) ON DELETE CASCADE. DeptCode здесь одновременно половина ключа и внешний ключ на владельца — именно это и делает Room слабой, а RoomNumber сам по себе частичный ключ. В виде DDL:

```
CREATE TABLE Room (
  DeptCode    CHAR(4),
  RoomNumber  VARCHAR(6),
  RoomType    VARCHAR(20) NOT NULL,
  BedCapacity SMALLINT CHECK (BedCapacity >= 0),
  Floor       SMALLINT,
  PRIMARY KEY (DeptCode, RoomNumber),
  FOREIGN KEY (DeptCode) REFERENCES Department(DeptCode) ON DELETE CASCADE
);
```

Такой ключ позволяет существовать и ('CARD','101'), и ('NEUR','101'), но дважды ('CARD','101') отвергает. Каскад — это зависимость по существованию.

Medication(MedicationID INT AUTO_INCREMENT, MedName VARCHAR(100) NOT NULL, Form VARCHAR(20) NOT NULL, Strength VARCHAR(30) NOT NULL, Manufacturer VARCHAR(80), UnitPrice DECIMAL(10,2) CHECK (UnitPrice >= 0), RequiresPrescription BOOLEAN NOT NULL DEFAULT TRUE) UNIQUE (MedName, Strength, Form).

Appointment(AppointmentID INT AUTO_INCREMENT, PatientID INT NOT NULL, DoctorID INT NOT NULL, AppointmentDateTime DATETIME NOT NULL, DurationMinutes SMALLINT NOT NULL DEFAULT 30, Purpose VARCHAR(200) NOT NULL, Notes TEXT NULL, Status VARCHAR(15) NOT NULL DEFAULT 'scheduled', RoomDeptCode CHAR(4) NULL, RoomNumber VARCHAR(6) NULL) Внешние ключи: PatientID -> Patient(PatientID) ON DELETE RESTRICT; DoctorID -> Doctor(DoctorID) ON DELETE RESTRICT; (RoomDeptCode, RoomNumber) -> Room(DeptCode, RoomNumber) ON DELETE SET NULL — составной внешний ключ, потому что у слабой сущности составной ключ.

На Appointment висят ещё UNIQUE (DoctorID, AppointmentDateTime) и UNIQUE (PatientID, AppointmentDateTime), те два альтернативных ключа, что выше. Занятость палаты закрывает UNIQUE (RoomDeptCode, RoomNumber, AppointmentDateTime); это UNIQUE по столбцам, допускающим NULL, и не потенциальный ключ: строки с NULL вместо палаты между собой не сталкиваются, а это как раз то, что нужно телефонным консультациям. CHECK держит два столбца палаты либо оба в NULL, либо оба заполненными, ведь составной внешний ключ не может присутствовать наполовину. EndTime не хранится. Правда, UNIQUE по времени начала не спасает от перекрытия: врач, записанный на 10:00 на 60 минут и ещё раз на 10:30 на 30 минут, проходит оба ограничения, так что эту часть нужно закрывать триггером.

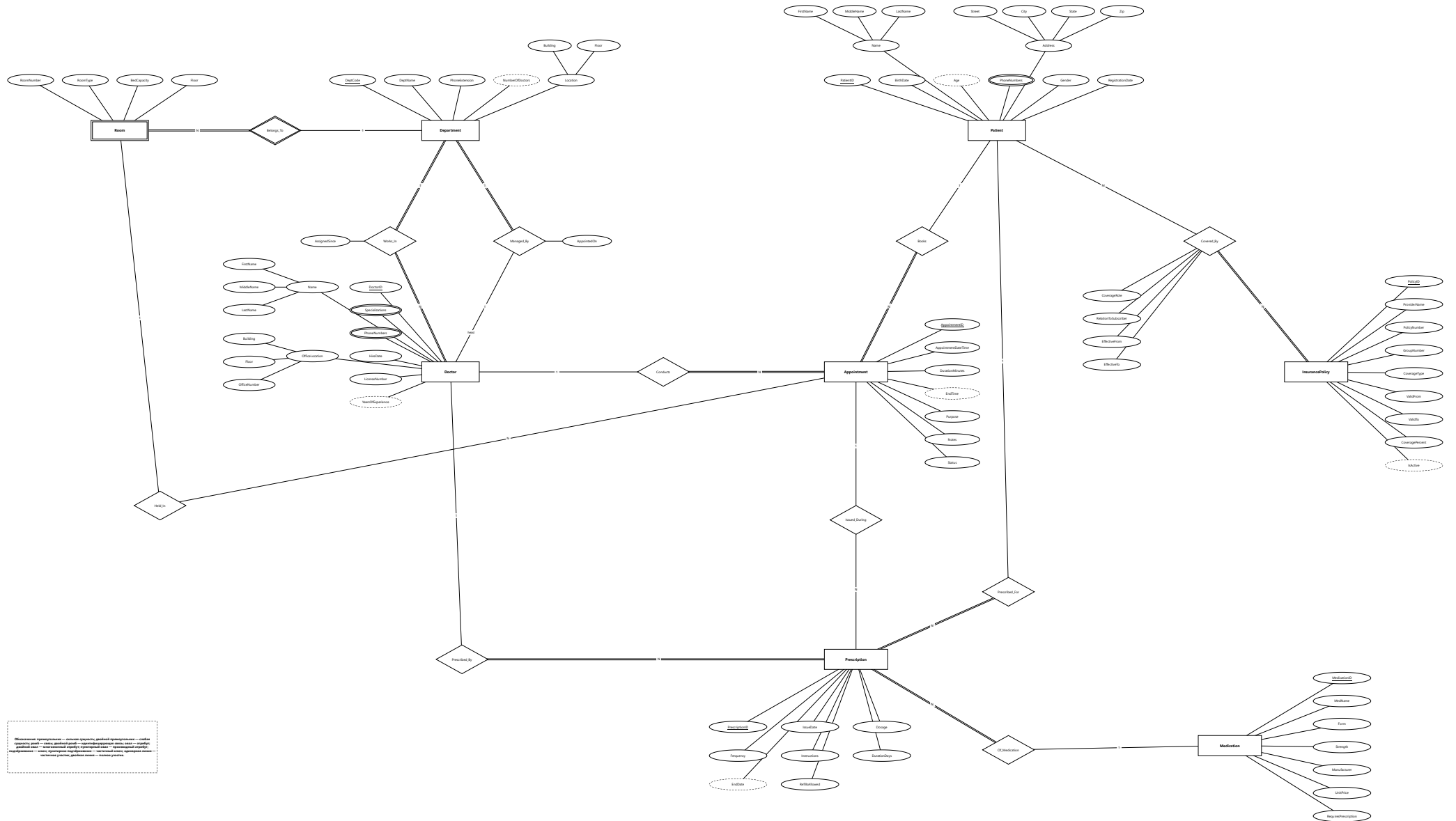
Prescription(PrescriptionID INT AUTO_INCREMENT, DoctorID INT NOT NULL, PatientID INT NOT NULL, MedicationID INT NOT NULL, AppointmentID INT NULL, IssueDate DATE NOT NULL DEFAULT CURRENT_DATE, Dosage VARCHAR(50) NOT NULL, Frequency VARCHAR(50) NOT NULL, Instructions TEXT NULL, DurationDays SMALLINT CHECK (DurationDays > 0), RefillsAllowed SMALLINT NOT NULL DEFAULT 0) Внешние ключи: DoctorID -> Doctor(DoctorID) ON DELETE RESTRICT; PatientID -> Patient(PatientID) ON DELETE RESTRICT; MedicationID -> Medication(MedicationID) ON DELETE RESTRICT; AppointmentID -> Appointment(AppointmentID) ON DELETE SET NULL. AppointmentID допускает NULL, потому что у телефонных продлений приёма нет. UNIQUE на (DoctorID, PatientID, MedicationID, IssueDate) намеренно не ставится, а EndDate не хранится.

Производные атрибуты возвращаются представлениями там, где они нужны, например CREATE VIEW v_Doctor AS SELECT d.*, TIMESTAMPDIFF(YEAR, HireDate, CURRENT_DATE) AS YearsOfExperience FROM Doctor d;

Два ограничения ER-модели до таблиц не дожили. У Department в Works_In участие полное, но внешний ключ со стороны Doctor умеет сказать там только NOT NULL, поэтому ничто не мешает завести отделение с нулём врачей. То же с полным участием InsurancePolicy в Covered_By: таблица-связка не может заставить родительскую строку в ней появиться. Для обоих понадобилось бы утверждение (assertion) или отложенный триггер. Есть и круговая зависимость: Department.HeadDoctorID должен быть NOT NULL, а Doctor.DeptCode уже такой, поэтому ни одну из строк нельзя вставить первой. Я сделал HeadDoctorID допускающим NULL, а правило «у каждого отделения есть заведующий» оставил проверке вне схемы, вместо ограничений DEFERRABLE INITIALLY DEFERRED, проверяемых на COMMIT. AppointedOn допускает NULL по той же причине.

Получается 12 таблиц: 7 от сильных сущностей, 1 от слабой, 3 от многозначных атрибутов и 1 таблица-связка.

Задание 2.1. Больница (нотация Чена)



Задание 2.2. Интернет-магазин

Я смоделировал интернет-магазин: шесть сильных сущностей, две слабые, одиннадцать связей. Я предположил, что товар лежит в одной категории, а покупатель оставляет не больше одного отзыва на товар.

Сущности

Сущность	Идентификатор (АК — альтернативный ключ)	Тип
CUSTOMER	CustomerID (AK Email)	сильная
PRODUCT	ProductID (AK SKU)	сильная
CATEGORY	CategoryID (AK CategoryName)	сильная
VENDOR	VendorID (AK TaxID)	сильная
ORDER	OrderID	сильная
WAREHOUSE	WarehouseID (AK WarehouseCode)	сильная
ADDRESS	(CustomerID, AddressNo), частичный ключ AddressNo	слабая, владелец CUSTOMER
ORDER_ITEM	(OrderID, LineNumber), частичный ключ LineNumber	слабая, владелец ORDER

Связи

Участие записано парой (min,max) для каждой стороны, в порядке перечисления: min 0 — частичное, min 1 — полное.

Связь	Между	Кардинальность	Участие
PLACES	CUSTOMER, ORDER	1:N	(0,N) : (1,1) полное
HAS_ADDRESS (идентифицирующая)	CUSTOMER, ADDRESS	1:N	(0,N) : (1,1) полное
SHIP_TO	ORDER, ADDRESS	N:1	(0,1) : (0,N), частичное с обеих сторон
BILL_TO	ORDER, ADDRESS	N:1	(0,1) : (0,N), частичное с обеих сторон
CONTAINS (идентифицирующая)	ORDER, ORDER_ITEM	1:N	(1,N) : (1,1), полное с обеих сторон
FOR_PRODUCT	ORDER_ITEM, PRODUCT	N:1	(1,1) полное : (0,N)
BELONGS_TO	PRODUCT, CATEGORY	N:1	(1,1) полное : (0,N)
SUBCATEGORY_OF (рекурсивная)	CATEGORY родитель, потомок	1:N	(0,N) : (0,1), частичное с обеих сторон
SUPPLIES	VENDOR, PRODUCT	M:N	(0,N) : (1,N) полное
REVIEWS	CUSTOMER, PRODUCT	M:N	(0,N) : (0,N), частичное с обеих сторон
STOCKED_AT	PRODUCT, WAREHOUSE	M:N	(0,N) : (0,N), частичное с обеих сторон

У SHIP_TO и BILL_TO участие со стороны заказа частичное: подарок едет по адресу, которого нет в адресной книге покупателя, поэтому указатель равен NULL и заполнен только снимок.

Ненпростые атрибуты

Составные: Customer.FullName (FirstName, MiddleName, LastName), Address.PostalAddress (Line1, Line2, City, Region, PostalCode, CountryCode), Product.Dimensions (LengthCm, WidthCm, HeightCm), Vendor.VendorAddress (Street, City, CountryCode), Warehouse.Location (City, CountryCode) и два снимка в ORDER, которые повторяют компоненты PostalAddress плюс Recipient. Region здесь потому, что от него зависит налоговая юрисдикция.

Многозначные: Product.Tags, Vendor.ContactPhone и Customer.Phone, который многозначный и составной одновременно (PhoneNumber, PhoneType).

Производные (пунктиром): Customer.Age, Product.AverageRating и ReviewCount, Order.OrderTotal (суммы строк плюс налог по каждой строке плюс ShippingFee), Order.ItemCount, OrderItem.LineTotal, QuantityAvailable. ShippingFee хранится, потому что он нужен для OrderTotal.

Слабые сущности

Сущность слабая, когда у неё нет собственного ключа, она не может существовать без конкретного владельца и идентифицируется только через связь с владельцем, так что её ключ — ключ владельца плюс частичный ключ. Одной зависимости по существованию мало: заказу ORDER нужен CUSTOMER, но OrderID опознаёт его глобально.

ORDER_ITEM — слабая сущность. LineNumber = 3 сам по себе ничего не значит, ведь номер 3 стоит у тысяч строк; он значит «строка 3 такого-то заказа», а после удаления этого заказа не значит вообще ничего. Полный ключ — (OrderID, LineNumber), CONTAINS рисуется двойным ромбом, ORDER_ITEM участвует как (1,1) полное, а ORDER как (1,N) полное, внешний ключ идёт с ON DELETE CASCADE — это та же зависимость, записанная на языке DDL. Дискриминатор — LineNumber, а не ProductID, потому что один товар может попасть в две строки одного заказа: часть по акции, часть по полной цене.

ADDRESS тоже слабая: AddressNo уникален только внутри адресной книги одного покупателя. REVIEW и INVENTORY выглядят похоже, но слабыми сущностями не являются: их ключи — только ключи участников, так что это M:N связи с атрибутами.

М:Н связи, которым нужны атрибуты

Связь	Атрибуты	Почему они относятся к паре
ORDER — PRODUCT, в виде ORDER_ITEM	Quantity, UnitPriceAtOrderTime, DiscountPct, TaxRatePct, ProductNameSnapshot, LineTotal (производный)	Сколько и по какой цене — свойство пары
REVIEWS	Rating, ReviewTitle, ReviewText, ReviewDate, VerifiedPurchase, HelpfulVotes	На PRODUCT оценка схлопывает всех авторов отзывов в одного, на CUSTOMER — все товары в один
SUPPLIES	SupplyPrice, LeadTimeDays, VendorSKU, IsPreferred	Один товар у разных поставщиков стоит по-разному, и у каждого свой код
STOCKED_AT	QuantityOnHand, QuantityReserved, QuantityAvailable, ReorderLevel, BinLocation, LastStockTake	Количества, точки дозаказа и коды ячеек у каждой площадки свои

Последние три нарисованы ромбами М:Н; связь Order — Product уже разрешена в ORDER_ITEM.

Почему цена живёт в строке заказа

Product.ListPrice определяется по ProductID, а цена, по которой выставлен счёт, определяется строкой: (OrderID, LineNumber) -> UnitPriceAtOrderTime. Разные детерминанты — разные отношения, поэтому выставленная цена не может лежать в PRODUCT.

Решающий довод исторический: заказ — неизменяемая финансовая запись. Если бы сумма счёта считалась как Quantity умножить на Product.ListPrice, то один UPDATE Product SET ListPrice = 899 WHERE ProductID = 42 незаметно переписал бы все прошлые счета по этому товару. Отчёты о выручке изменились бы задним числом, счёт перестал бы сходиться с тем, что списал платёжный сервис, возвраты считались бы по текущей цене, а не по уплаченной, а сданную налоговую отчётность нельзя было бы воспроизвести.

Цена не функция товара даже в один момент времени: купоны, уровни лояльности и договорные цены позволяют двум покупателям заказать одно и то же в одну минуту и заплатить разное. То же касается TaxRatePct, Currency, ProductNameSnapshot и адресов.

Адрес оплаты и адрес доставки

Две группы адресных столбцов в CUSTOMER — это повторяющаяся группа примерно из четырнадцати столбцов, по большей части с NULL, с местом ровно под один адрес доставки и без истории. Столбец AddressType в единственной сущности ADDRESS немногим лучше: тип — это роль, которую адрес играет в сделке, а не его свойство.

Поэтому я сделал ADDRESS слабой сущностью CUSTOMER, отдельные связи SHIP_TO и BILL_TO от ORDER и полный снимок обоих адресов в заказе. Роль вынесена на связь, поэтому одна строка адреса годится сразу для обеих ролей и для любого числа заказов, а снимок хранит историю: покупатель переехал и поправил адрес 2 — это не должно менять то, куда ушла прошлогодняя посылка.

Здесь окупается составной ключ. Ключ ADDRESS — (CustomerID, AddressNo), а в заказе уже есть CustomerID, поэтому внешний ключ (CustomerID, ShipAddressNo) -> ADDRESS(CustomerID, AddressNo) переиспользует этот столбец, и доставка на чужой адрес становится невозможной.

Складские остатки

Столбцы с количеством в PRODUCT посадили бы самый горячий на запись столбец системы на строку каталога, которую в основном читают, и всё равно не сказали бы, где лежит товар; отдельная таблица INVENTORY в 1:1 с PRODUCT слепа ровно так же. Я взял сущность WAREHOUSE и М:Н связь STOCKED_AT с ключом (ProductID, WarehouseID), потому что рано или поздно появится второй склад или пункт возвратов, а плоским схемам тогда нужна миграция и переписывание каждого запроса, который трогает остатки. Пока склад один, всё работает как разделение 1:1, ценой одного соединения.

QuantityAvailable = QuantityOnHand - QuantityReserved — производный атрибут, поэтому это вычисляемый столбец, а не третий столбец, который может разбегаться с остальными. Именно QuantityReserved не даёт продать лишнего в промежутке между оформлением заказа и его сборкой.

Реляционная схема

Подчёркнутое — первичный ключ, курсив — внешний ключ, то и другое сразу — столбец ключа, который заодно внешний. ORDER в SQL зарезервировано, отсюда CUSTOMER_ORDER.

CUSTOMER(CustomerID INT, FirstName VARCHAR(50) NOT NULL, MiddleName VARCHAR(50) NULL, LastName VARCHAR(50) NOT NULL, Email VARCHAR(255) NOT NULL, PasswordHash CHAR(60) NOT NULL, DateOfBirth DATE NULL, RegistrationDate DATETIME NOT NULL, AccountStatus VARCHAR(20) NOT NULL), UNIQUE(Email)

CUSTOMER_PHONE(CustomerID INT, PhoneNumber VARCHAR(20), PhoneType VARCHAR(10) NULL)

ADDRESS(CustomerID INT, AddressNo SMALLINT, Label VARCHAR(30) NULL, Recipient VARCHAR(100) NOT NULL, Line1 VARCHAR(100) NOT NULL, Line2 VARCHAR(100) NULL, City VARCHAR(60) NOT NULL, Region VARCHAR(60) NULL, PostalCode VARCHAR(12) NOT NULL, CountryCode CHAR(2) NOT NULL, IsDefaultBilling BOOLEAN NOT NULL DEFAULT FALSE, IsDefaultShipping BOOLEAN NOT NULL DEFAULT FALSE, IsArchived BOOLEAN NOT NULL DEFAULT FALSE). Слабая сущность, вместо удаления помечается удалённой.

CATEGORY(CategoryID INT, CategoryName VARCHAR(60) NOT NULL, Description TEXT NULL, ParentCategoryID INT NULL), UNIQUE(CategoryName), CHECK (ParentCategoryID <> CategoryID). NULL в родителе означает корневую категорию.

VENDOR(VendorID INT, VendorName VARCHAR(100) NOT NULL, TaxID VARCHAR(20) NOT NULL, ContactEmail VARCHAR(255) NOT NULL, Street VARCHAR(100) NULL, City VARCHAR(60) NULL, CountryCode CHAR(2) NULL), UNIQUE(TaxID)

VENDOR_PHONE(VendorID INT, PhoneNumber VARCHAR(20))

PRODUCT(ProductID INT, SKU CHAR(12) NOT NULL, ProductName VARCHAR(150) NOT NULL, Description TEXT NULL, ListPrice DECIMAL(10,2) NOT NULL, WeightKg DECIMAL(7,3) NULL, LengthCm DECIMAL(6,2) NULL, WidthCm DECIMAL(6,2) NULL, HeightCm DECIMAL(6,2) NULL, DateAdded DATE NOT NULL,

IsActive BOOLEAN NOT NULL DEFAULT TRUE, CategoryID INT NOT NULL), UNIQUE(SKU), CHECK (ListPrice >= 0). ListPrice — текущая цена каталога; AverageRating и ReviewCount берутся из представления.

PRODUCT_TAG(ProductID INT, Tag VARCHAR(30))

WAREHOUSE(WarehouseID INT, WarehouseCode CHAR(4) NOT NULL, WarehouseName VARCHAR(80) NOT NULL, City VARCHAR(60) NOT NULL, CountryCode CHAR(2) NOT NULL, CapacityM3 DECIMAL(10,2) NULL), UNIQUE(WarehouseCode)

SUPPLIES(VendorID INT, ProductID INT, SupplyPrice DECIMAL(10,2) NOT NULL, LeadTimeDays SMALLINT NOT NULL, VendorSKU VARCHAR(40) NULL, IsPreferred BOOLEAN NOT NULL DEFAULT FALSE), CHECK (SupplyPrice >= 0 AND LeadTimeDays >= 0)

INVENTORY(ProductID INT, WarehouseID INT, QuantityOnHand INT NOT NULL DEFAULT 0, QuantityReserved INT NOT NULL DEFAULT 0, ReorderLevel INT NOT NULL DEFAULT 0, BinLocation VARCHAR(20) NULL, LastStockTake DATETIME NULL), CHECK (QuantityOnHand >= 0 AND QuantityReserved >= 0 AND QuantityReserved <= QuantityOnHand). Так отображается связь STOCKED_AT, производное количество вычисляется:

```
ALTER TABLE INVENTORY ADD COLUMN QuantityAvailable INT
GENERATED ALWAYS AS (QuantityOnHand - QuantityReserved) STORED;
```

CUSTOMER_ORDER(OrderID BIGINT, CustomerID INT NOT NULL, OrderDate DATETIME NOT NULL, OrderStatus VARCHAR(20) NOT NULL, PaymentMethod VARCHAR(20) NOT NULL, Currency CHAR(3) NOT NULL, ShippingFee DECIMAL(10,2) NOT NULL DEFAULT 0, ShipAddressNo SMALLINT NULL, BillAddressNo SMALLINT NULL, TrackingNumber VARCHAR(40) NULL, плюс две группы снимков ShipTo... и BillTo..., в каждой Recipient VARCHAR(100) NOT NULL, Line1 VARCHAR(100) NOT NULL, Line2 VARCHAR(100) NULL, City VARCHAR(60) NOT NULL, Region VARCHAR(60) NULL, PostalCode VARCHAR(12) NOT NULL, CountryCode CHAR(2) NOT NULL), CHECK (ShippingFee >= 0). Оба адресных внешних ключа составные и переиспользуют CustomerID; указатели могут быть NULL, снимок — нет. OrderTotal и ItemCount берутся из представления.

ORDER_ITEM(OrderID BIGINT, LineNumber SMALLINT, ProductID INT NOT NULL, ProductNameSnapshot VARCHAR(150) NOT NULL, Quantity INT NOT NULL, UnitPriceAtOrderTime DECIMAL(10,2) NOT NULL, DiscountPct DECIMAL(5,2) NOT NULL DEFAULT 0, TaxRatePct DECIMAL(5,2) NOT NULL DEFAULT 0), CHECK (Quantity > 0), CHECK (UnitPriceAtOrderTime >= 0), CHECK (DiscountPct BETWEEN 0 AND 100). Слабая сущность. LineTotal вычисляется как ROUND(Quantity * UnitPriceAtOrderTime * (1 - DiscountPct/100), 2), округление идёт по каждой строке, а сумма считается уже после, потому что при обратном порядке остаются расхождения на уровне центов.

REVIEW(CustomerID INT, ProductID INT, Rating SMALLINT NOT NULL, ReviewTitle VARCHAR(120) NULL, ReviewText TEXT NULL, ReviewDate DATETIME NOT NULL, VerifiedPurchase BOOLEAN NOT NULL DEFAULT FALSE, HelpfulVotes INT NOT NULL DEFAULT 0), CHECK (Rating BETWEEN 1 AND 5). Ключ и есть правило «один отзыв от покупателя на один товар».

Внешние ключи

Все внешние ключи идут с ON UPDATE CASCADE.

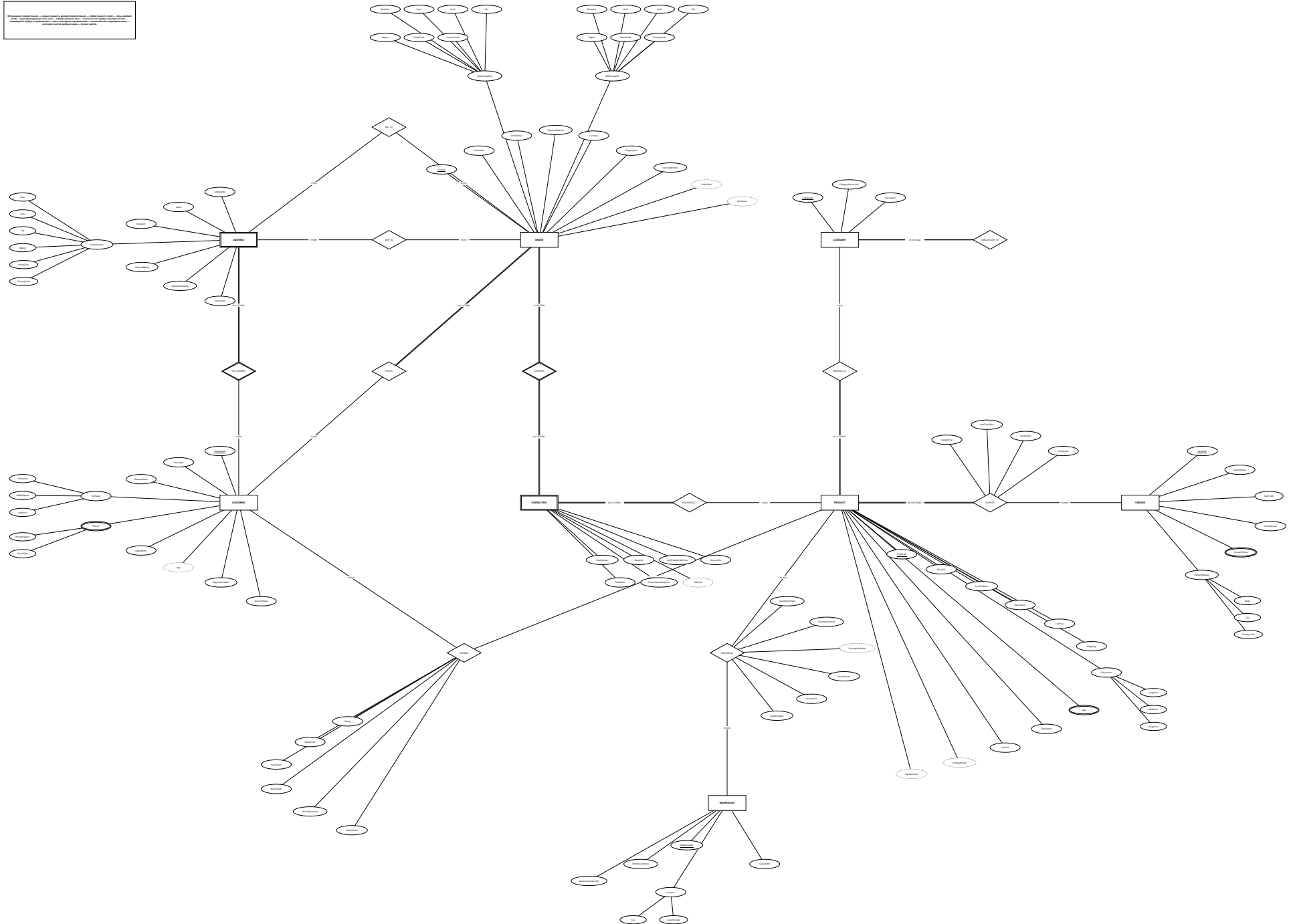
Потомок (столбцы)	Родитель	ON DELETE
CUSTOMER_PHONE.CustomerID	CUSTOMER.CustomerID	CASCADE
ADDRESS.CustomerID	CUSTOMER.CustomerID	CASCADE
CATEGORY.ParentCategoryID	CATEGORY.CategoryID	SET NULL
PRODUCT.CategoryID	CATEGORY.CategoryID	RESTRICT
PRODUCT_TAG.ProductID	PRODUCT.ProductID	CASCADE
VENDOR_PHONE.VendorID	VENDOR.VendorID	CASCADE
SUPPLIES.VendorID	VENDOR.VendorID	CASCADE
SUPPLIES.ProductID	PRODUCT.ProductID	CASCADE
INVENTORY.ProductID	PRODUCT.ProductID	CASCADE
INVENTORY.WarehouseID	WAREHOUSE.WarehouseID	RESTRICT
CUSTOMER_ORDER.CustomerID	CUSTOMER.CustomerID	RESTRICT
CUSTOMER_ORDER.(CustomerID, ShipAddressNo)	ADDRESS.(CustomerID, AddressNo)	RESTRICT
CUSTOMER_ORDER.(CustomerID, BillAddressNo)	ADDRESS.(CustomerID, AddressNo)	RESTRICT
ORDER_ITEM.OrderID	CUSTOMER_ORDER.OrderID	CASCADE
ORDER_ITEM.ProductID	PRODUCT.ProductID	RESTRICT
REVIEW.CustomerID	CUSTOMER.CustomerID	CASCADE
REVIEW.ProductID	PRODUCT.ProductID	CASCADE

RESTRICT стоит потому, что финансовая запись должна пережить удаление покупателя, товара или склада, на который она ссылается; их вместо этого обезличивают или помечают IsActive = FALSE. Два правила внешним ключом не выразить вовсе: «в заказе есть хотя бы одна строка» и «у каждого товара есть хотя бы один поставщик», ведь родитель должен существовать раньше, чем на него сошлётся первый потомок. Обоим нужно отложенное ограничение или триггер на момент фиксации.

Нормализация

Составные атрибуты стали атомарными столбцами, а многозначные — отдельными отношениями (1НФ), причём телефоны держатся отдельно от адресов (4НФ). CategoryName нет в PRODUCT: иначе получилось бы ProductID -> CategoryID -> CategoryName и аномалия при переименовании. Каждый

детерминант — потенциальный ключ, поэтому все четырнадцать отношений в НФБК, и снимки этого не ломают: родитель может измениться, а потомок нет.



Часть 4. Нормализация

Задание 4.1. Нормализация StudentProject до 3НФ

Отношение, которое я нормализую:

```
StudentProject(StudentID, StudentName, StudentMajor,
               ProjectID, ProjectTitle, ProjectType,
               SupervisorID, SupervisorName, SupervisorDept,
               Role, HoursWorked, StartDate, EndDate)
```

Функциональные зависимости (ФЗ)

Бизнес-правила, которые я принял: StudentID определяет студента, у которого одно имя и одна заявленная специальность. ProjectID определяет проект, у которого одно название, один тип и один руководитель (руководитель может вести несколько проектов, поэтому Supervises — это 1:N). SupervisorID определяет преподавателя с одним именем и одной кафедрой. Студент может работать над несколькими проектами, а в проекте несколько студентов, поэтому ни один из этих ID сам по себе строку не определяет. Внутри одного проекта у студента одна роль и одно общее количество часов.

Ещё я предполагаю, что StartDate и EndDate относятся к проекту, а не к участию конкретного студента в нём. У лабораторного или дипломного проекта один административно закреплённый срок, общий для всех участников, да и столбцы названы StartDate и EndDate, а не JoinDate и LeaveDate.

```
F1: StudentID      -> StudentName, StudentMajor
F2: ProjectID      -> ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate
F3: SupervisorID   -> SupervisorName, SupervisorDept
F4: (StudentID, ProjectID) -> Role, HoursWorked
```

Ещё одна зависимость следует отсюда по транзитивности: из F2 получаем ProjectID -> SupervisorID, из F3 — SupervisorID -> SupervisorName, SupervisorDept, значит, ProjectID -> SupervisorName, SupervisorDept. Она выводится, а не задана, поэтому в F не входит, и именно её уберёт третья нормальная форма (3НФ, 3NF).

Три зависимости выглядят правдоподобно, но не выполняются. StudentName -> StudentID не работает, потому что два студента могут носить одно имя, даже если в примере ниже такого и не случилось. ProjectTitle -> ProjectID не работает, потому что названия не уникальны: два набора могут вести проект «Library RFID System». StudentID -> Role тоже не выполняется: S001 — Developer в P10 и Team Lead в P20.

Избыточность и аномалии

Пример данных:

StudentID	StudentName	StudentMajor	ProjectID	ProjectTitle	ProjectType	SupervisorID	SupervisorName	SupervisorDept	Role	HoursWorked
S001	Aisha Nurlanova	Computer Science	P10	Campus Navigation App	Research	F21	Marat Ospanov	Software Engineering	Developer	120
S002	Daniyar Seitov	Information Systems	P10	Campus Navigation App	Research	F21	Marat Ospanov	Software Engineering	Tester	80
S003	Kamila Abenova	Computer Science	P10	Campus Navigation App	Research	F21	Marat Ospanov	Software Engineering	Analyst	95
S001	Aisha Nurlanova	Computer Science	P20	IoT Greenhouse Monitor	Course	F35	Assel Bekova	Computer Engineering	Team Lead	60
S004	Timur Zhaksylykov	Mathematics	P20	IoT Greenhouse Monitor	Course	F35	Assel Bekova	Computer Engineering	Developer	140
S002	Daniyar Seitov	Information Systems	P30	Library RFID System	Diploma	F21	Marat Ospanov	Software Engineering	Developer	200

Всё описание P10 (название, тип, руководитель, обе даты) выписано трижды, по разу на каждого участника, а описание P20 — дважды. Пара («Marat Ospanov», «Software Engineering») стоит в четырёх строках, а имя и специальность студента появляются заново каждый раз, когда он подключается к очередному проекту. Только Role и HoursWorked — действительно факты о строке. Всё остальное — факт о студенте, проекте или руководителе, скопированный по разу на каждое участие.

Это копирование и даёт три обычные аномалии.

Аномалия обновления. Если F21 сменит фамилию на Ospanova, мне придётся править четыре строки. Пропущу одну или транзакция оборвётся на половине — и таблица будет утверждать, что F21 одновременно Ospanov и Ospanova. Схема этому никак не мешает: ни одно ограничение не говорит, что SupervisorID определяет SupervisorName.

Аномалия вставки. Допустим, кафедра нанимает руководителя F44 (Nurlan Kadyrov, Applied Mathematics), у которого ещё нет проекта, или регистрирует проект P40 до того, как записался хоть один студент. Ни тот, ни другой факт записать некуда. Ключ — (StudentID, ProjectID), так что у строки часть ключа была бы NULL, а это запрещено целостностью сущностей.

Аномалия удаления. Если S002 уйдёт из P30 и я удалю шестую строку, то потеряю единственную запись о том, что P30 «Library RFID System» вообще существует, а вместе с ней тип, даты и руководителя. Удаление участия тихо удаляет проект.

Первая нормальная форма

Каждый столбец здесь хранит одно скалярное значение. Нет ни списка ролей в ячейке, ни перечня студентов, втиснутого в StudentID, ни повторяющейся группы Project1/Project2. Таблица уже в 1НФ, чинить нечего, и отношение в 1НФ — это исходное отношение с составным ключом, который доказан в следующем разделе.

```
StudentProject_1NF(StudentID, StudentName, StudentMajor,
                   ProjectID, ProjectTitle, ProjectType,
                   SupervisorID, SupervisorName, SupervisorDept,
                   Role, HoursWorked, StartDate, EndDate)
```

Первичный ключ — StudentID, ProjectID.

Столбец	Тип	Столбец	Тип
<u>StudentID</u>	CHAR(6)	SupervisorID	CHAR(4)
StudentName	VARCHAR(60)	SupervisorName	VARCHAR(60)
StudentMajor	VARCHAR(50)	SupervisorDept	VARCHAR(50)
<u>ProjectID</u>	CHAR(4)	Role	VARCHAR(30)
ProjectTitle	VARCHAR(100)	HoursWorked	DECIMAL(6,2)
ProjectType	VARCHAR(20)	StartDate, EndDate	DATE

Она в 1НФ, но не в 2НФ.

Вторая нормальная форма

Первичный ключ

Ключ я нашёл через замыкания атрибутов. Замыкание {StudentID} — это только StudentID, StudentName, StudentMajor, три атрибута из тринадцати. Замыкание {ProjectID} — восемь атрибутов: столбцы проекта плюс SupervisorID, а через F3 ещё SupervisorName и SupervisorDept. Ни то, ни другое не суперключ.

Замыкание {StudentID, ProjectID} — все тринадцать. F1 добавляет два столбца студента, F2 — столбцы проекта и SupervisorID, F3 — два столбца руководителя, F4 — Role и HoursWorked. Значит, это суперключ, а так как ни одна из половин суперключом не была, выбросить из него нельзя ничего, и это потенциальный ключ. Ни StudentID, ни ProjectID никогда не стоят в правой части ФЗ, поэтому никакое замыкание их не даст и оба входят в каждый потенциальный ключ. Остаётся единственный потенциальный ключ {StudentID, ProjectID}, он же первичный. StudentID и ProjectID — ключевые атрибуты, остальные одиннадцать неключевые.

Наборы вроде {StudentID, ProjectID, Role} — тоже суперключи, но не потенциальные ключи: Role можно убрать, и свойство суперключа сохранится.

Частичные зависимости

Частичная зависимость — это когда неключевой атрибут зависит от части составного ключа. Таких три.

#	Зависимость	Детерминант	Вывод
P1	StudentID -> StudentName, StudentMajor	половина ключа	частичная, нарушает 2НФ
P2	ProjectID -> ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate	половина ключа	частичная, нарушает 2НФ
P3	ProjectID -> SupervisorName, SupervisorDept (выводится)	половина ключа	частичная, нарушает 2НФ

С F4 всё в порядке: Role и HoursWorked зависят от всего ключа. F3 вообще не частичная, потому что SupervisorID в ключ не входит. Она транзитивная, и с ней разбирается 3НФ.

Декомпозиция

Я разбил отношение по детерминантам.

```
STUDENT_2NF(StudentID, StudentName, StudentMajor)
PROJECT_2NF(ProjectID, ProjectTitle, ProjectType,
             SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate)
PARTICIPATION_2NF(StudentID, ProjectID, Role, HoursWorked)
```

STUDENT_2NF

Столбец	Тип	Примечание
<u>StudentID</u>	CHAR(6)	PK
StudentName	VARCHAR(60)	NOT NULL
StudentMajor	VARCHAR(50)	NOT NULL

PROJECT_2NF

Столбец	Тип	Примечание
<u>ProjectID</u>	CHAR(4)	PK
ProjectTitle	VARCHAR(100)	NOT NULL
ProjectType	VARCHAR(20)	Research, Course или Diploma
SupervisorID	CHAR(4)	на этом этапе обычный столбец
SupervisorName	VARCHAR(60)	по-прежнему избыточен
SupervisorDept	VARCHAR(50)	по-прежнему избыточен
StartDate	DATE	NOT NULL
EndDate	DATE	

PARTICIPATION_2NF

Столбец	Тип	Примечание
<u>StudentID</u>	CHAR(6)	часть PK, FK на STUDENT_2NF
<u>ProjectID</u>	CHAR(4)	часть PK, FK на PROJECT_2NF
Role	VARCHAR(30)	NOT NULL
HoursWorked	DECIMAL(6,2)	не меньше 0

Внешние ключи на этом этапе: PARTICIPATION_2NF.StudentID на STUDENT_2NF.StudentID и PARTICIPATION_2NF.ProjectID на PROJECT_2NF.ProjectID.

Теперь все три в 2НФ. У первых двух ключ из одного атрибута, так что частичная зависимость там невозможна, а в третьей оба неключевых столбца зависят от полного ключа. PROJECT_2NF по-прежнему не в 3НФ.

Третья нормальная форма

Ломает её зависимость SupervisorID -> SupervisorName, SupervisorDept внутри PROJECT_2NF. SupervisorID не ключ этой таблицы (F21 ведёт P10 и P30), а два столбца, которые он определяет, неключевые, так что это транзитивная зависимость по цепочке ProjectID -> SupervisorID -> SupervisorName, SupervisorDept. Симптом виден до сих пор: («Marat Ospanov», «Software Engineering») хранится в PROJECT_2NF дважды, то есть ослабленная аномалия обновления пережила 2НФ. В STUDENT_2NF и PARTICIPATION_2NF транзитивных зависимостей нет, потому что в обеих единственный детерминант — сам ключ.

Лечится это так: SupervisorID и два зависящих от него столбца выносятся в отдельную таблицу, а в PROJECT SupervisorID остаётся внешним ключом.

STUDENT

Столбец	Тип данных	Ограничение
<u>StudentID</u>	CHAR(6)	PRIMARY KEY
StudentName	VARCHAR(60)	NOT NULL
StudentMajor	VARCHAR(50)	NOT NULL

SUPERVISOR

Столбец	Тип данных	Ограничение
<u>SupervisorID</u>	CHAR(4)	PRIMARY KEY
SupervisorName	VARCHAR(60)	NOT NULL
SupervisorDept	VARCHAR(50)	NOT NULL

PROJECT

Столбец	Тип данных	Ограничение
<u>ProjectID</u>	CHAR(4)	PRIMARY KEY
ProjectTitle	VARCHAR(100)	NOT NULL
ProjectType	VARCHAR(20)	CHECK IN ('Research','Course','Diploma')
<i>SupervisorID</i>	CHAR(4)	NOT NULL, FK на SUPERVISOR
StartDate	DATE	NOT NULL
EndDate	DATE	CHECK (EndDate >= StartDate)

PARTICIPATION

Столбец	Тип данных	Ограничение
<u>StudentID</u>	CHAR(6)	часть PK, FK на STUDENT
<u>ProjectID</u>	CHAR(4)	часть PK, FK на PROJECT
Role	VARCHAR(30)	NOT NULL
HoursWorked	DECIMAL(6,2)	DEFAULT 0, CHECK (HoursWorked >= 0)

В линейной форме, первичные ключи подчёркнуты, внешние — курсивом:

STUDENT(StudentID, StudentName, StudentMajor) SUPERVISOR(SupervisorID, SupervisorName, SupervisorDept) PROJECT(ProjectID, ProjectTitle, ProjectType, *SupervisorID*, StartDate, EndDate) PARTICIPATION(*StudentID*, *ProjectID*, Role, HoursWorked)

#	Дочерняя таблица	Родительская таблица	Кратность	ON DELETE
FK1	PROJECT.SupervisorID	SUPERVISOR.SupervisorID	много проектов к одному руководителю	RESTRICT
FK2	PARTICIPATION.StudentID	STUDENT.StudentID	много участий к одному студенту	CASCADE
FK3	PARTICIPATION.ProjectID	PROJECT.ProjectID	много участий к одному проекту	CASCADE

Все четыре таблицы в 3НФ, и трёх аномалий больше нет. Имя и кафедра руководителя хранятся один раз, так что исправление фамилии — это один UPDATE. Руководитель без проектов — одна строка в SUPERVISOR, проект без студентов — одна строка в PROJECT, и NULL в ключе не нужен ни там, ни там. Удаление (S002, P30) из PARTICIPATION убирает только это участие, а P30 и F21 остаются.

Декомпозиция даёт соединение без потерь: при каждом разбиении общий атрибут (сначала StudentID, потом ProjectID, потом SupervisorID) — первичный ключ одного из двух фрагментов, поэтому обратное соединение четырёх таблиц вернёт исходные шесть строк и ничего лишнего. Зависимости она тоже сохраняет: F1 лежит в STUDENT, F2 — в PROJECT, F3 — в SUPERVISOR, F4 — в PARTICIPATION, так что каждую можно проверить внутри одной таблицы, без соединения.

Заодно результат оказался и в НФБК. В каждой таблице ровно один потенциальный ключ, и он же единственный оставшийся в ней детерминант, так что неключевых детерминантов нигде нет и придирается НФБК не к чему.

Задание 4.2. Нормализация CourseSchedule до НФБК

Универсальное отношение:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

Атрибуты я сокращаю до SID, Maj, CID, CNm, IID, INm, TS, Rm, Bld.

#	ФЗ	Бизнес-правило, из которого она следует
f1	SID -> Maj	у каждого студента ровно одна специальность
f2	CID -> CNm	у каждого курса фиксированное название
f3	IID -> INm	у каждого преподавателя ровно одно имя
f4	Rm -> Bld	номера аудиторий уникальны в пределах всего кампуса
f5	CID -> IID, TS, Rm	поток ведёт один преподаватель, в одно время, в одной аудитории
f6	Rm, TS -> CID	два потока не могут занимать одну аудиторию в один и тот же слот
f7	IID, TS -> CID	преподаватель не может быть в двух местах сразу (моё допущение)

Три из них потребовали решения. Четвёртое правило сформулировано как «каждый временной слот в аудитории определяет здание», то есть буквально это (Rm, TS) -> Bld. Но в том же предложении сказано, что аудитории уникальны по всему кампусу, значит здание определяется одной только аудиторией, и я беру Rm -> Bld. Более слабая форма следует отсюда по правилу пополнения, так что ничего не теряется. f5 выполняется, только если CourseID определяет один поток, а не курс из каталога, и читаю я его именно так: будь у CSCI151 два потока, CID -> TS было бы неверно. f7 в условии не задана. Я её предполагаю и дальше везде провожу явно.

Если сцепить заданные ФЗ, получаются выводимые, и это не самостоятельные факты: CID -> Bld (f5, затем f4), CID -> INm (f5, затем f3), Rm, TS -> CNm, IID, INm, Bld (f6, затем f2, f5, f3, f4) и IID, TS -> CNm, Rm, Bld.

f5 вместе с f6 делают CID и (Rm, TS) эквивалентными, а f5 вместе с f7 — CID и (IID, TS). У этих трёх детерминантов одинаковые замыкания, поэтому внутри ключа любой из них может заменить остальные.

Потенциальные ключи и первичный ключ

SID не встречается в правой части ни одной ФЗ, поэтому породить его не может никакая ФЗ, а значит, он входит в каждый потенциальный ключ. Сам по себе он не ключ: {SID}+ = {SID, Maj}. Значит, любой потенциальный ключ — это SID плюс минимальный набор, замыкание которого достаёт до CID: как только CID внутри, остальное подтягивается само.

{SID, CID}+

Шаг	Использованная ФЗ	Набор на этом шаге
0	-	{SID, CID}
1	f1	{SID, CID, Maj}
2	f2	+ CNm
3	f5	+ IID, TS, Rm
4	f3	+ INm
5	f4	+ Bld, и замыкание равно R

Минимальность: {CID}+ = {CID, CNm, IID, INm, TS, Rm, Bld} — это 7 атрибутов из 9, нет SID и Maj; {SID}+ = {SID, Maj}. Значит, CK1 = {StudentID, CourseID}.

{SID, Rm, TS}+

Шаг	Использованная ФЗ	Набор на этом шаге
0	-	{SID, Rm, TS}
1	f1	+ Maj
2	f4	+ Bld
3	f6	+ CID
4	f2	+ CNm
5	f5	+ IID
6	f3	+ INm, и замыкание равно R

Минимальность: {Rm, TS}+ = {Rm, TS, Bld, CID, CNm, IID, INm}, без SID и Maj; {SID, TS}+ = {SID, Maj, TS}; {SID, Rm}+ = {SID, Maj, Rm, Bld}. Значит, CK2 = {StudentID, Room, TimeSlot}.

{SID, IID, TS}+

Шаг	Использованная ФЗ	Набор на этом шаге
0	-	{SID, IID, TS}
1	f1	+ Maj
2	f3	+ INm
3	f7	+ CID
4	f2	+ CNm
5	f5	+ Rm
6	f4	+ Bld, и замыкание равно R

Минимальность: {IID, TS}+ = {IID, TS, INm, CID, CNm, Rm, Bld}; {SID, TS}+ = {SID, Maj, TS}; {SID, IID}+ = {SID, Maj, IID, INm}. Значит, CK3 = {StudentID, InstructorID, TimeSlot}.

Их ровно три. В каждый потенциальный ключ входит SID, а при наличии SID замыкание доходит до всего отношения, как только становится достижим CID. Детерминанты CID — только сам CID, (Rm, TS) и (IID, TS). Больше ничего не помогает: Maj, CNm, INm и Bld не определяют ничего, один TS не определяет ничего, один Rm даёт только Bld, один IID — только INm.

Ключевые атрибуты: SID, CID, Rm, TS, IID. Неключевые: Maj, CNm, INm, Bld.

Суперключ — это любое множество, замыкание которого равно R; потенциальный ключ — это суперключ, ни одно собственное подмножество которого не суперключ. (SID, CID, TimeSlot) и (SID, Room, TimeSlot, Building) — суперключи, но не потенциальные ключи: убери TimeSlot или Building, суперключ всё равно останется.

Первичным ключом я выбрал (StudentID, CourseID). Он самый короткий из трёх, и обе его части — устойчивые идентификаторы, тогда как Room и TimeSlot меняются в течение семестра чаще всего. С CK2 перенос в другую аудиторию менял бы первичный ключ и каскадом расходился по всем ссылающимся строкам. К тому же он прямо говорит, что записано в строке: один студент, записанный на один поток. CK2 и CK3 этот выбор не выбрасывает. После декомпозиции они следуют из UNIQUE(Room, TimeSlot) и UNIQUE(InstructorID, TimeSlot) в Section вместе с ключом Enrollment.

Ключ получился неуклюжий, потому что отношение склеивает два факта разной степени детализации: факт записи (SID с CID) и факт расписания (CID, IID, TS, Rm, Bld). Шесть ФЗ из семи о студентах вообще ничего не говорят. Если читать схему бегло, кажется, что ключом должно быть (CourseID, TimeSlot) или (Room, TimeSlot). Но строка — это один студент на одном потоке, ни один из этих наборов не определяет StudentID, а сам StudentID не стоит ни в одной правой части, так что каждый потенциальный ключ приходится им дополнять. А эквивалентность CID, (Rm, TS) и (IID, TS) превращает этот единственный шаблон в три ключа.

Находится ли отношение в НФБК?

Нет, и даже не в 2НФ. Все четыре неключевых атрибута зависят от собственного подмножества какого-нибудь потенциального ключа: Maj от SID, который входит во все три ключа; CNm, а по выводимости INm и Bld — от CID, части CK1; Bld от Rm, части CK2; INm от IID, части CK3. Как только нарушена 2НФ, третья нормальная форма (3НФ, 3NF) и НФБК нарушаются автоматически. Обе проверки всё же стоит провести: они показывают, какие ФЗ ведут декомпозицию.

По ЗНФ правило нарушают f1, f2, f3 и f4: слева не суперключ, справа неключевой атрибут. Остальные три проходят, но только потому, что их правые части случайно оказались ключевыми атрибутами. В этом зазоре и вся разница между ЗНФ и НФБК.

НФБК, проверка по каждой ФЗ:

ФЗ	Замыкание левой части	Суперключ?	Нарушение
SID -> Maj	{SID, Maj}	нет	да
CID -> CNm, IID, TS, Rm	{CID, CNm, IID, INm, TS, Rm, Bld}	нет, не хватает SID и Maj	да
IID -> INm	{IID, INm}	нет	да
Rm -> Bld	{Rm, Bld}	нет	да
Rm, TS -> CID	{Rm, TS, Bld, CID, CNm, IID, INm}	нет, не хватает SID и Maj	да
IID, TS -> CID	{IID, TS, INm, CID, CNm, Rm, Bld}	нет, не хватает SID и Maj	да
SID, CID -> всё (то же для CK2, CK3)	R	да	нет

Каждая нетривиальная неключевая ФЗ нарушает НФБК. Ни одна левая часть, кроме суперключа с SID внутри, до SID не добирается, так что у четырёх справочных ФЗ и у ФЗ расписания внутри универсального отношения нет шансов. Аномалии те же три, что обычно: StudentMajor повторяется по разу на каждый поток, куда записан студент; поток, на который никто не записан, сохранить вообще негде; а удаление последней записи на поток уничтожает его аудиторию, время и преподавателя.

Декомпозиция до НФБК

На каждом шаге я беру нарушающую ФЗ $X \rightarrow Y$ и разбиваю отношение надвое: замыкание X , ограниченное этим отношением, и X плюс все оставшиеся атрибуты. Разбиваю всегда по полному замыканию X , а не по одному атрибуту правой части, потому что именно так теряются зависимости.

Шаг 1, по f1 ($SID \rightarrow Maj$), где $\{SID\}^+ = \{SID, Maj\}$:

- $R1 = \{SID, Maj\}$ — будущая Student. Единственная её ФЗ — $SID \rightarrow Maj$, и SID здесь ключ, так что она в НФБК.
- $R2 = \{SID, CID, CNm, IID, INm, TS, Rm, Bld\}$.

Шаг 2. В R2 ключи по-прежнему CK1, CK2 и CK3. Нарушающая ФЗ — $CID \rightarrow CNm, IID, INm, TS, Rm, Bld$:

- $R21 = \{CID, CNm, IID, INm, TS, Rm, Bld\}$.
- $R22 = \{SID, CID\}$ — будущая Enrollment. Нетривиальных ФЗ у неё нет, ключ (SID, CID), так что она в НФБК.

Шаг 3. В R21 ключи — CID, (Rm, TS) и (IID, TS). Остаются два нарушения: $IID \rightarrow INm$ и $Rm \rightarrow Bld$. Беру первое:

- $R211 = \{IID, INm\}$ — будущая Instructor, ключ IID, в НФБК.
- $R212 = \{CID, CNm, IID, TS, Rm, Bld\}$.

Шаг 4, по $Rm \rightarrow Bld$, замыкание которой внутри R212 — это {Rm, Bld}, и суперключом там оно не служит:

- $R2121 = \{Rm, Bld\}$ — будущая RoomLocation, ключ Rm, в НФБК.
- $R2122 = \{CID, CNm, IID, TS, Rm\}$ — будущая Section.

Шаг 5, проверка Section. Спроецированные на неё ФЗ — это $CID \rightarrow CNm, IID, TS, Rm$, а также $Rm, TS \rightarrow CID$ и $IID, TS \rightarrow CID$. Все три левые части — потенциальные ключи Section, нарушения нет, и алгоритм останавливается. $Rm \rightarrow Bld$ и $IID \rightarrow INm$ больше не применимы, потому что Bld и INm ушли.

$R(SID, Maj, CID, CNm, IID, INm, TS, Rm, Bld)$	
Student(SID, Maj)	разбиение по $SID \rightarrow Maj$
$R2(SID, CID, CNm, IID, INm, TS, Rm, Bld)$	
Enrollment(SID, CID)	разбиение по $CID \rightarrow$ замыкание CID
$R21(CID, CNm, IID, INm, TS, Rm, Bld)$	
Instructor(IID, INm)	разбиение по $IID \rightarrow INm$
$R212(CID, CNm, IID, TS, Rm, Bld)$	
RoomLocation(Rm, Bld)	разбиение по $Rm \rightarrow Bld$
Section(CID, CNm, IID, TS, Rm)	

Итоговая схема, первичные ключи подчеркнуты, внешние ключи курсивом:

1. Student(StudentID, StudentMajor)
2. Instructor(InstructorID, InstructorName)
3. RoomLocation(Room, Building)
4. Section(CourseID, CourseName, *InstructorID*, TimeSlot, Room)
5. Enrollment(StudentID, CourseID)

Отношение	Атрибут	Тип	Ограничение
Student	StudentID	CHAR(9)	PK
Student	StudentMajor	VARCHAR(50)	NOT NULL
Instructor	InstructorID	INT	PK
Instructor	InstructorName	VARCHAR(60)	NOT NULL
RoomLocation	Room	VARCHAR(10)	PK
RoomLocation	Building	VARCHAR(40)	NOT NULL
Section	CourseID	VARCHAR(10)	PK
Section	CourseName	VARCHAR(100)	NOT NULL
Section	InstructorID	INT	FK, NOT NULL, UNIQUE(InstructorID, TimeSlot)
Section	TimeSlot	VARCHAR(20)	NOT NULL, входит в оба UNIQUE
Section	Room	VARCHAR(10)	FK, NOT NULL, UNIQUE(Room, TimeSlot)
Enrollment	StudentID	CHAR(9)	часть PK, FK
Enrollment	CourseID	VARCHAR(10)	часть PK, FK

Внешние ключи:

FK	Дочерняя	Родительская	Действие
FK1	Section.InstructorID	Instructor.InstructorID	ON DELETE RESTRICT
FK2	Section.Room	RoomLocation.Room	ON DELETE RESTRICT
FK3	Enrollment.StudentID	Student.StudentID	ON DELETE CASCADE
FK4	Enrollment.CourseID	Section.CourseID	ON DELETE CASCADE

Самая интересная здесь Section. Она сохраняет все три пересекающихся потенциальных ключа, и, поскольку каждый детерминант сам по себе ключ, она попадает в НФБК, а не только в ЗНФ.

Что теряется

На уровне кортежей — ничего. Разбиение надвое даёт соединение без потерь, если общие атрибуты образуют ключ хотя бы одной из сторон, и здесь это выполняется на каждом шаге по построению: общие атрибуты — всегда X, а X — ключ той стороны, где лежит замыкание.

Разбиение	Общие атрибуты	Ключ чего
1	{SID}	{SID, Maj}, по f1
2	{CID}	{CID, CNm, IID, INm, TS, Rm, Bld}, по f5 вместе с f2, f3, f4
3	{IID}	{IID, INm}, по f3
4	{Rm}	{Rm, Bld}, по f4

Свойство отсутствия потерь сохраняется при композиции разбиений, поэтому обратное соединение Student, Enrollment, Section, Instructor и RoomLocation воспроизводит CourseSchedule для любого допустимого экземпляра, без лишних кортежей.

Зависимости тоже выживают. Проекция F на пять отношений: Student держит f1, Instructor — f3, RoomLocation — f4, Section — f2, f5, f6 и f7, а Enrollment не держит ничего. Это все семь, и обеспечиваются они первичными ключами плюс UNIQUE(Room, TimeSlot) и UNIQUE(InstructorID, TimeSlot) в Section, так что объединение проекций эквивалентно F.

Хочется сказать, что жертвой пала (Room, TimeSlot) -> CourseID, но нет. Room, TimeSlot и CourseID остаются вместе внутри Section, поэтому двойная бронь аудитории отсекается обычным ограничением UNIQUE, без всякого триггера. Это следует из отмеченной выше эквивалентности: Rm, TS и IID лежат внутри замыкания CID, поэтому шаг 2 уносит их в один и тот же фрагмент.

Настоящая потеря появляется, если добавить ещё одно правило, которого в условии нет. Допустим, университет также требует, чтобы студент не посещал два потока в один слот, то есть f8: SID, TS -> CID. Тогда {SID, TS}⁺ = R, значит (SID, TS) становится потенциальным ключом, а СК2 и СК3 потенциальными ключами быть перестают: каждый из них строго содержит {SID, TS} и больше не минимален. При F вместе с f8 потенциальных ключей ровно два: (SID, CID) и (SID, TS). В декомпозированной схеме SID встречается только в Student и Enrollment, а TS — только в Section, так что ни одно отношение не держит SID, TS и CID вместе, и f8 не обеспечить ни ключом, ни ограничением UNIQUE. Её придётся делать процедурно:

```
CREATE ASSERTION no_student_clash CHECK (NOT EXISTS (
  SELECT e1.StudentID, s1.TimeSlot
  FROM Enrollment e1 JOIN Section s1 ON e1.CourseID = s1.CourseID
  GROUP BY e1.StudentID, s1.TimeSlot HAVING COUNT(*) > 1));
```

Вот настоящая цена НФБК. Ф3, атрибуты которой оказались по разные стороны разбиения, превращается в проверку во время выполнения, подверженную гонкам при конкурентном доступе, вместо ограничения, которое обеспечивает индекс.

Вариант с ЗНФ и что я бы взял в работу

F с одноатрибутными правыми частями — уже минимальное покрытие из девяти ФЗ. Ни у f6, ни у f7 нет лишнего атрибута слева, а удаление любой ФЗ сжимает замыкание её левой части так, что правая в него уже не попадает. Синтез группирует ФЗ по левой части, добавляет (SID, CID) как отношение, достраивающее ключ, и выбрасывает группы (Rm, TS, CID) и (IID, TS, CID), потому что обе сидят внутри группы CID. Получаются те же пять отношений, что выдал алгоритм НФБК, так что обычной платы здесь не требуется. В общем случае он платится: синтез в ЗНФ всегда сохраняет зависимости, но допускает избыточность, а НФБК избыточность убирает и при этом может разорвать ФЗ и потерять её. Здесь НФБК не стоит ничего, потому что Room, TimeSlot, InstructorID и CourseID лежат внутри замыкания CourseID и никогда не расходятся. Я бы взял в работу схему в НФБК, приведённую выше. При ФЗ с f1 по f7 она одновременно в НФБК, даёт соединение без потерь и сохраняет зависимости. Если f8 примут позже, я оставил бы эту схему и добавил бы ASSERTION, а не откатывался бы к варианту в ЗНФ, который хранит факт «студент, слот, поток» дважды.

Допущения, которые стоит отметить

- CourseID определяет поток, а не курс из каталога. Если университет запускает по нескольку потоков одного курса, f5 неверна, и Section распадается на Course(CourseCode CHAR(8), CourseName VARCHAR(100)) и Section(SectionID INT, CourseCode CHAR(8), InstructorID INT, TimeSlot VARCHAR(20), Room VARCHAR(10)), а Enrollment ссылается на SectionID. Это по-прежнему НФБК, и оба ограничения UNIQUE остаются как есть.
 - f7 — моя, а не из условия. Если преподавателя можно указывать сразу на нескольких одновременных потоках, f7 убирается, СКЗ исчезает, вместе с ним уходит UNIQUE(InstructorID, TimeSlot), и больше в декомпозиции ничего не меняется.
 - TimeSlot как VARCHAR(20) трактует слот как непрозрачный код вроде 'MON-09:00-10:50'. Если слоты нужно сравнивать или проверять на пересечение, он должен превратиться в TimeSlot(SlotID INT, DayOfWeek CHAR(3), StartTime TIME, EndTime TIME) с внешним ключом из Section. Это вопрос 1НФ, отдельный от всей работы с НФБК выше.
-

Часть 5. Проектирование

Задание 5.1. Студенческие клубы и организации

В задании несколько вещей оставлены на усмотрение, поэтому сначала я их закрепил:

- На каждого студента и каждый клуб приходится одна строка членства: повторное вступление открывает ту же строку заново, поэтому истории повторных вступлений нет.
- У каждого клуба ровно один куратор из преподавателей, и дату назначения я записываю. Преподаватель может курировать много клубов или ни одного.
- У срока полномочий есть дата начала и необязательная дата окончания, два срока по одной должности в одном клубе не могут пересекаться, а должностное лицо обязано состоять в этом клубе.
- Должности берутся из каталога общеуниверситетских названий плюс из тех, что клуб заводит сам, и клуб может занять только глобальную должность или свою собственную.
- Каждое мероприятие принадлежит ровно одному клубу. Бронь называет аудиторию и клуб-заказчика, но называть мероприятие не обязана: клубы бронируют ещё и репетиции, и время про запас. Две одобренные брони на одну аудиторию не могут пересекаться.
- Один бюджет на клуб и финансовый год, каждый расход отнесён к одному бюджету, а остаток не хранится никогда. Финансовый год и учебный год — одно и то же, и определяются они один раз в общем календаре.
- Посещаемость ведётся по студенту и мероприятию, статус идёт от Invited через Registered к Attended, NoShow или Declined. Не члены клуба могут приходить на мероприятия, помеченные как открытые для них.

Правила о пересечениях, правило области действия должности и правило «только для членов» затрагивают по несколько таблиц каждое, поэтому это триггеры или ограничения исключения.

Сущности

Сущность	Идентификатор	Тип
Student	StudentID	сильная
Faculty	FacultyID	сильная
Club	ClubID	сильная
Position	PositionID	сильная, каталог
Event	EventID	сильная
Room	RoomID	сильная
RoomReservation	ReservationID	сильная, чтобы аудиторию можно было придержать до появления мероприятия
AcademicCalendar	AcademicYear	сильная, справочник
Membership	(StudentID, ClubID)	ассоциативная
OfficerTerm	(ClubID, PositionID, StartDate)	слабая, дискриминатор StartDate
Attendance	(StudentID, EventID)	ассоциативная
Budget	(ClubID, FiscalYear)	слабая, владелец Club
Expense	(ClubID, FiscalYear, ExpenseNo)	слабая, владелец Budget
Income	(ClubID, FiscalYear, IncomeNo)	слабая, владелец Budget

Ключ Membership и Attendance целиком собран из ключей владельцев, так что строго говоря это ассоциативные сущности, а не слабые, тогда как OfficerTerm, Budget, Expense и Income добавляют каждая свой дискриминатор. Все шесть нарисованы двойными прямоугольниками на двойных ромбах. Membership нужна именно в такой форме, а не обычным ромбом связи M:N, потому что OfficerTerm должна цепляться к самому факту членства.

Связи

Связь	Между	Кардинальность	Участие
Advises	Faculty, Club	1:N	Club полное, Faculty частичное
EnrolledAs	Student, Membership	1:N	Membership полное, Student частичное
HasMember	Club, Membership	1:N	Membership полное, Club частичное
OfficerIn	Club, OfficerTerm	1:N	Term полное, Club частичное
TermOf	Position, OfficerTerm	1:N	Term полное, Position частичное
FilledBy	Membership, OfficerTerm	1:N	Term полное, Membership частичное
Defines	Club, Position	1:N	оба частичные
Organizes	Club, Event	1:N	Event полное, Club частичное
RegisteredVia	Student, Attendance	1:N	Attendance полное, Student частичное
HasAttendance	Event, Attendance	1:N	Attendance полное, Event частичное
HostedIn	Event, RoomReservation	1:N	оба частичные
Books	Room, RoomReservation	1:N	Reservation полное, Room частичное
RequestedBy	Club, RoomReservation	1:N	Reservation полное, Club частичное
HasBudget	Club, Budget	1:N	Budget полное, Club частичное
ChargedTo	Budget, Expense	1:N	Expense полное, Budget частичное
CreditedTo	Budget, Income	1:N	Income полное, Budget частичное
IncurredFor	Event, Expense	1:N	оба частичные
CoversYear	AcademicCalendar, Budget	1:N	Budget полное, календарь частичное
ApprovedBy	Faculty, Budget	1:N	оба частичные

Advises — единственная связь с атрибутом, это AssignedDate. Частичные концы тут важны: бронь или расход могут не относиться ни к какому мероприятию, глобальная должность не принадлежит ни одному клубу, а у неутверждённого бюджета нет утвердившего.

Непростые атрибуты

Составных четыре: Student.Name (FirstName, MiddleName, LastName), Faculty.Name, Faculty.OfficeLocation (OfficeBuilding, OfficeRoom) и Room.Location (Building, RoomNumber), который заодно и альтернативный ключ. Многозначных два: Club.MeetingDay, ведь клуб может собираться во вторник и в пятницу, и Student.Phone, который многозначный и составной сразу, у каждого значения есть PhoneNumber, различающий значения, и PhoneType.

Производные — это Student.Age, Event.Duration, Position.Scope, Membership.TenureDays, OfficerTerm.IsCurrent и AcademicYear, Budget.TotalSpent, TotalIncome и RemainingBalance, а также счётчики у Student, Club, Faculty и Event. Ни один не хранится, это представления, и ради этого их и рисуют пунктирными овалами.

Реляционная схема

Student(StudentID CHAR(9), FirstName VARCHAR(50) NOT NULL, MiddleName VARCHAR(50) NULL, LastName VARCHAR(50) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, DateOfBirth DATE NOT NULL, Major VARCHAR(60), EnrollmentYear SMALLINT NOT NULL). Email — альтернативный ключ; столбца Age нет.

StudentPhone(StudentID CHAR(9), PhoneNumber VARCHAR(20), PhoneType VARCHAR(10) NOT NULL DEFAULT 'Mobile'). Внешний ключ: StudentID references Student(StudentID) ON DELETE CASCADE.

Faculty(FacultyID CHAR(8), FirstName VARCHAR(50) NOT NULL, LastName VARCHAR(50) NOT NULL, Department VARCHAR(60) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, OfficeBuilding VARCHAR(40), OfficeRoom VARCHAR(10)).

Club(ClubID INT, ClubName VARCHAR(80) NOT NULL UNIQUE, Category VARCHAR(30) NOT NULL, FoundedDate DATE NOT NULL, Description TEXT, Status VARCHAR(10) NOT NULL DEFAULT 'Active', AdvisorID CHAR(8) NOT NULL, AdvisorAssignedDate DATE NOT NULL). Внешний ключ: AdvisorID references Faculty(FacultyID) ON DELETE RESTRICT. AdvisorID NOT NULL кодирует полное участие Club в Advises; AdvisorAssignedDate — собственный атрибут этой связи, и находиться он обязан на стороне N.

ClubMeetingDay(ClubID INT, DayOfWeek VARCHAR(9)). Внешний ключ: ClubID references Club(ClubID) ON DELETE CASCADE.

AcademicCalendar(AcademicYear CHAR(9), YearStart DATE NOT NULL, YearEnd DATE NOT NULL). UNIQUE (YearStart); UNIQUE (YearEnd), поскольку годы разбивают временную ось без пересечений, а значит каждая граница задаёт год однозначно.

Position(PositionID INT, Title VARCHAR(40) NOT NULL, Duties TEXT, PrecedenceRank SMALLINT NOT NULL DEFAULT 99, ClubID INT NULL). Внешний ключ: ClubID references Club(ClubID) ON DELETE CASCADE. UNIQUE (ClubID, Title) плюс частичный индекс UNIQUE (Title) WHERE ClubID IS NULL, потому что два NULL в SQL не равны друг другу и один составной UNIQUE пропустил бы две глобальные строки «Treasurer». ClubID IS NULL помечает общеуниверситетскую должность.

Membership(StudentID CHAR(9), ClubID INT, JoinDate DATE NOT NULL, LeaveDate DATE NULL, MembershipStatus VARCHAR(10) NOT NULL DEFAULT 'Active'). Внешние ключи: StudentID references Student(StudentID) ON DELETE CASCADE; ClubID references Club(ClubID) ON DELETE CASCADE. Ключ здесь естественный, а не суррогатный MembershipID, чтобы на него могла указывать OfficerTerm.

OfficerTerm(ClubID INT, PositionID INT, StartDate DATE, StudentID CHAR(9) NOT NULL, EndDate DATE NULL, ElectionMethod VARCHAR(15) NOT NULL DEFAULT 'Elected'). Внешние ключи: ClubID references Club(ClubID); PositionID references Position(PositionID); (StudentID, ClubID) references Membership(StudentID, ClubID) ON DELETE RESTRICT. Пересечение сроков запрещает EXCLUDE USING gist (ClubID WITH =, PositionID WITH =, daterange(StartDate, COALESCE(EndDate, DATE '9999-12-31')) WITH &&).

Этот составной внешний ключ стоит здесь не зря: один столбец ClubID называет и клуб срока полномочий, и клуб членства, поэтому должностное лицо клуба X доказуемо состоит в клубе X. В ключ таблицы я взял StartDate, а не учебный год, чтобы президента, сложившего полномочия в декабре, можно было сменить в январе; AcademicYear здесь не хранится вовсе, ведь StartDate -> AcademicYear была бы зависимостью от части ключа и сломала бы 2НФ. Правило области действия для должностей приходится делать триггером: оба столбца ключа NOT NULL, поэтому внешний ключ никогда не совпал бы с глобальной должностью, у которой ClubID NULL.

Event(EventID INT, ClubID INT NOT NULL, Title VARCHAR(120) NOT NULL, EventDate DATE NOT NULL, StartTime TIME NOT NULL, EndTime TIME NOT NULL, EventType VARCHAR(20) NOT NULL, Description TEXT, MaxCapacity INT NULL, OpenToNonMembers BOOLEAN NOT NULL DEFAULT TRUE, Status VARCHAR(10) NOT NULL DEFAULT 'Planned'). Внешний ключ: ClubID references Club(ClubID) ON DELETE CASCADE. UNIQUE (EventID, ClubID) — чтобы две таблицы ниже могли сослаться на эту пару.

Attendance(StudentID CHAR(9), EventID INT, RSVPStatus VARCHAR(12) NOT NULL DEFAULT 'Invited', CheckInTime TIMESTAMP NULL, AttendeeRole VARCHAR(12) NOT NULL DEFAULT 'Participant', FeedbackRating SMALLINT NULL). Внешние ключи: StudentID references Student(StudentID) ON DELETE CASCADE; EventID references Event(EventID) ON DELETE CASCADE.

Room(RoomID INT, Building VARCHAR(40) NOT NULL, RoomNumber VARCHAR(10) NOT NULL, Capacity INT NOT NULL, RoomType VARCHAR(20) NOT NULL, HasProjector BOOLEAN NOT NULL DEFAULT FALSE, IsAccessible BOOLEAN NOT NULL DEFAULT TRUE). UNIQUE (Building, RoomNumber) — второй потенциальный ключ; первичным я сделал RoomID, потому что при ремонте аудитории перенумеровывают.

RoomReservation(ReservationID INT, RoomID INT NOT NULL, ClubID INT NOT NULL, EventID INT NULL, ReservationDate DATE NOT NULL, StartTime TIME NOT NULL, EndTime TIME NOT NULL, RequestedOn TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, Status VARCHAR(10) NOT NULL DEFAULT 'Requested', Purpose VARCHAR(120)). Внешние ключи: RoomID references Room(RoomID) ON DELETE RESTRICT; ClubID references Club(ClubID) ON DELETE CASCADE; (EventID, ClubID) references Event(EventID, ClubID) ON DELETE SET NULL (EventID).

Составной внешний ключ держит бронь привязанной только к мероприятиям того клуба, который её запросил, а ссылочное действие при удалении ограничено одним столбцом, потому что обычный SET NULL попробовал бы обнулить заодно и ClubID, и тогда удаление любого мероприятия падало бы. Двойное бронирование запрещает EXCLUDE USING gist (RoomID WITH =, tsrange(ReservationDate + StartTime, ReservationDate + EndTime) WITH &&) WHERE (Status = 'Approved'), которому нужно расширение btree_gist, как и ограничению на OfficerTerm. Дата свёрнута в диапазон, потому что в PostgreSQL нет диапазонного типа над временем, а ограничение правила одними одобренными строками позволяет двум клубам запросить один и тот же слот, тогда как получит его только один.

Budget(ClubID INT, FiscalYear CHAR(9), AllocatedAmount DECIMAL(10,2) NOT NULL DEFAULT 0.00, ApprovedDate DATE NULL, ApprovedBy CHAR(8) NULL). Внешние ключи: ClubID references Club(ClubID) ON DELETE CASCADE; FiscalYear references AcademicCalendar(AcademicYear) ON DELETE RESTRICT; ApprovedBy references Faculty(FacultyID) ON DELETE RESTRICT, допускает NULL, потому что у неутверждённого бюджета нет утвердившего. Столбца RemainingBalance нет.

Expense(ClubID INT, FiscalYear CHAR(9), ExpenseNo INT, ExpenseDate DATE NOT NULL, Amount DECIMAL(10,2) NOT NULL, Category VARCHAR(20) NOT NULL, Description VARCHAR(200), Vendor VARCHAR(80), ReceiptRef VARCHAR(120), ApprovalStatus VARCHAR(10) NOT NULL DEFAULT 'Pending', EventID INT NULL). Внешние ключи: (ClubID, FiscalYear) references Budget(ClubID, FiscalYear) ON DELETE CASCADE; (EventID, ClubID) references Event(EventID, ClubID) ON DELETE SET NULL (EventID) — снова ограничение одним столбцом, и здесь оно необходимо, потому что ClubID входит в первичный ключ.

Income(ClubID INT, FiscalYear CHAR(9), IncomeNo INT, IncomeDate DATE NOT NULL, Amount DECIMAL(10,2) NOT NULL, Source VARCHAR(20) NOT NULL, Description VARCHAR(200)). Внешний ключ: (ClubID, FiscalYear) references Budget(ClubID, FiscalYear) ON DELETE CASCADE.

На столбцах статуса, категории и типа висят CHECK по спискам допустимых значений, суммы проверяются на положительность, а каждая пара дат или времён проверяется так, чтобы конец шёл после начала.

Производные значения несут четыре представления: v_Student добавляет Age, v_ClubBalance добавляет TotalSpent, TotalIncome и RemainingBalance = AllocatedAmount + TotalIncome - TotalSpent, v_EventRegistrations считает регистрации по каждому мероприятию, а v_OfficerTermYear показывает учебный год срока полномочий.

Нормальные формы

На уровне схемы каждое отношение в НФБК. В отношениях с ключом из одного атрибута частичная зависимость невозможна в принципе, а транзитивные я убрал руками: имена кураторов живут в Faculty, вместимость аудитории — в Room, дата мероприятия не копируется в Attendance, а выделенная сумма не копируется в каждую строку расхода.

Два отношения требуют пояснения. И Expense, и RoomReservation несут EventID и ClubID вместе, чтобы составной внешний ключ мог гарантировать: строка привязана к мероприятию того же клуба. На строках, где EventID не NULL, это даёт EventID -> ClubID, а EventID не суперключ ни в одном из этих отношений, так что на таком подмножестве НФБК нарушается.

Плохи они по-разному. Ключ Expense — (ClubID, FiscalYear, ExpenseNo), поэтому ClubID ключевой атрибут, и зависимость здесь как раз то исключение, которое третья нормальная форма (3НФ, 3NF) прямо разрешает: на этом подмножестве Expense в 3НФ, но не в НФБК. У RoomReservation ключ — один ReservationID, значит ClubID там неключевой, а ReservationID -> EventID -> ClubID — обычная транзитивная зависимость: на этом подмножестве отношение опускается до 2НФ.

Столбец я всё равно оставил. Бронь обязана называть запросивший её клуб даже тогда, когда мероприятия ещё нет, ради этого EventID и допускает NULL, а составной внешний ключ не даёт двум столбцам противоречить друг другу, поэтому аномалия обновления, ради которой и существует 3НФ, возникнуть не может. Стоит заметить и другое: функциональная зависимость (ФЗ) по идее должна выполняться на всём отношении, а эта ничего не говорит о строках с NULL в EventID, так что при обычном прочтении она не выполняется вовсе и оба отношения в НФБК. Выше приведено более строгое прочтение по подмножеству строк, потому что из двух оно менее лестное.

Проектные решения

Должности: enum, справочник или столбец в Membership

Проще всего был бы enum: CHECK (Title IN ('President','Treasurer','Secretary')) на строке должностного лица, никакой лишней таблицы. Но тогда роли живут в схеме, и добавление «Social Media Manager» означает миграцию, а обязанностям и старшинству вообще некуда деться. Столбец Role в Membership ещё хуже: он несёт только текущую роль, история сроков пропадает, и один студент не может занимать две должности.

Я выбрал справочник Position с необязательным ClubID. Роли становятся данными, клуб может завести свою должность прямо во время работы, а ClubID IS NULL отделяет общеуниверситетский каталог от клубных выдумок. Плата — лишнее соединение, частичный уникальный индекс, к которому вынуждают NULL, и правило области действия, которое приходится выносить в триггер.

Обязано ли должностное лицо быть членом клуба

OfficerTerm могла бы ссылаться прямо на Student. Внешний ключ проще, и так можно было бы завести почётное должностное лицо со стороны, но правилу тогда нужны триггеры на вставку в OfficerTerm и на удаление из Membership, а они зависят от диалекта и легко обходятся массовой загрузкой. Ссылка на Membership с добавлением JoinDate сохранила бы ещё и историю повторных вступлений, ценой внешнего ключа из трёх столбцов в каждом запросе про должностных лиц.

Я сделал так, что OfficerTerm ссылается на Membership по (StudentID, ClubID): должностное лицо клуба X, не состоящее в X, просто невыразимо, и правило ничего не стоит во время работы. Отказаться пришлось от суррогатного ключа у Membership и от любой записи о том, что студент вступил повторно.

Остаток бюджета: хранимый столбец или расчёт по книге операций

Если хранить RemainingBalance в Budget и поддерживать его триггерами, самый частый вопрос читается за константное время. Но это дублирует книгу операций: каждая вставка расхода, каждая смена статуса утверждения и каждая правка задним числом обязаны обновить его в той же транзакции, и один пропущенный путь тихо оставит остаток неверным.

Я взял чистую книгу операций и считаю остаток по требованию в v_ClubBalance. Тогда он никогда не разойдётся с чеками, а у RemainingBalance в системе одно определение, AllocatedAmount + TotalIncome - TotalSpent, и его обязан воспроизводить каждый отчёт. Плата — сканирование с агрегацией на каждый запрос. Если станет медленно, индекс на Expense(ClubID, FiscalYear, ApprovalStatus, Amount) это лечит и модель не трогает.

Примеры запросов

Q1. Вывести всех студентов, которые сейчас занимают должность в клубе Computer Science Club, с названием должности и датами срока. Пять таблиц, фильтр по названию клуба и по тому, что срок открыт сегодня.

```
SELECT  s.StudentID,
        s.FirstName || ' ' || s.LastName AS OfficerName,
        p.Title AS Position, ot.StartDate, ot.EndDate
FROM    Club c
JOIN    OfficerTerm ot ON ot.ClubID = c.ClubID
JOIN    Position p ON p.PositionID = ot.PositionID
JOIN    Membership m ON m.StudentID = ot.StudentID
        AND m.ClubID = ot.ClubID
JOIN    Student s ON s.StudentID = m.StudentID
WHERE   c.ClubName = 'Computer Science Club'
        AND ot.StartDate <= CURRENT_DATE
        AND (ot.EndDate IS NULL OR ot.EndDate >= CURRENT_DATE)
ORDER BY p.PrecedenceRank, s.LastName;
```

Q2. Найти все активные клубы, где на текущий учебный год ни один студент не записан как President. NOT EXISTS по OfficerTerm, соединённой с Position и с календарём, откуда и берётся учебный год.

```
SELECT  c.ClubID, c.ClubName
FROM    Club c
WHERE   c.Status = 'Active'
        AND NOT EXISTS (
            SELECT 1
            FROM    OfficerTerm ot
            JOIN    Position p ON p.PositionID = ot.PositionID
            JOIN    AcademicCalendar ac ON ot.StartDate BETWEEN ac.YearStart
                                                AND ac.YearEnd
            WHERE   ot.ClubID = c.ClubID
                    AND p.Title = 'President'
                    AND CURRENT_DATE BETWEEN ac.YearStart AND ac.YearEnd
        )
ORDER BY c.ClubName;
```

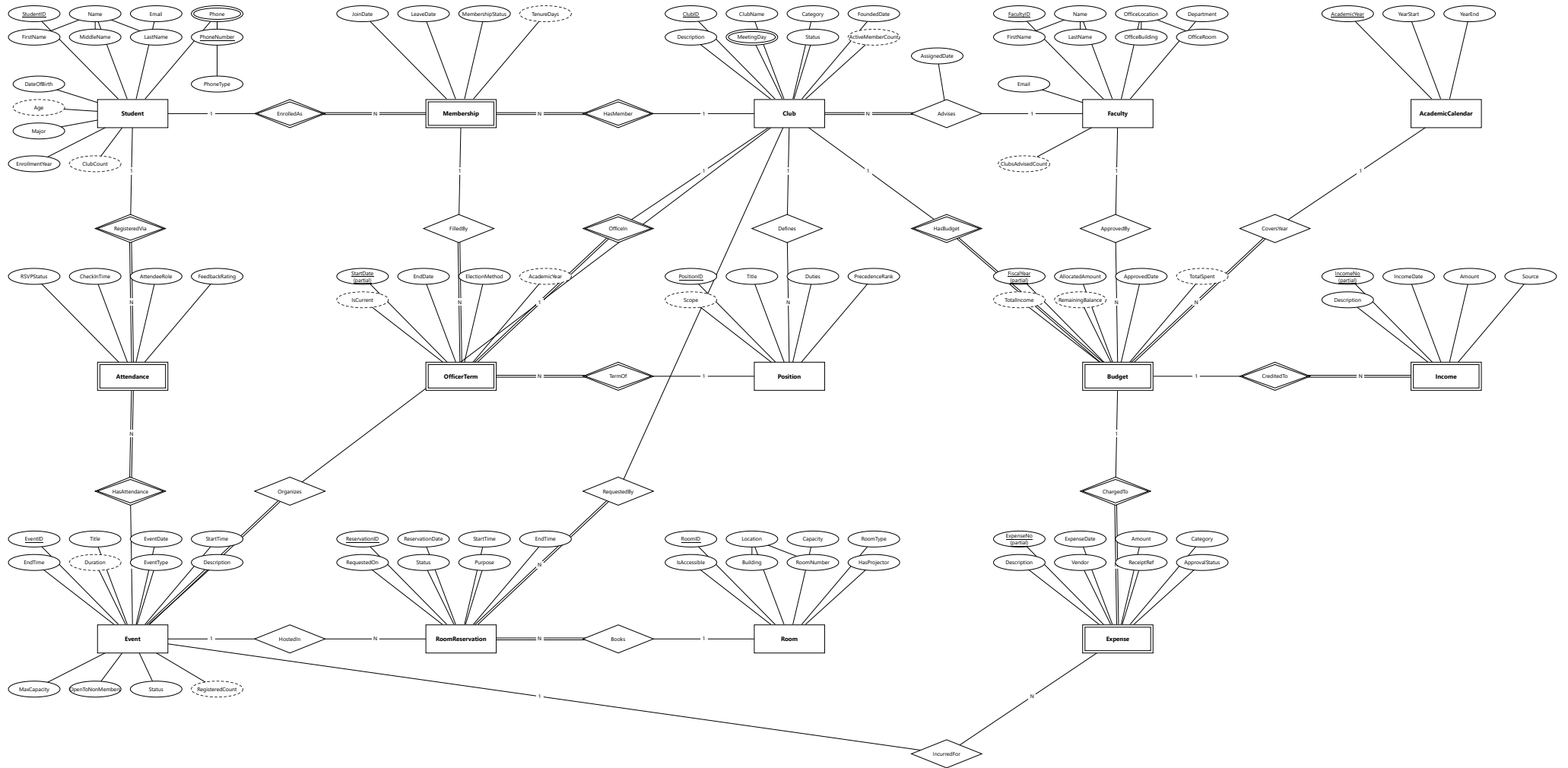
Q3. Найти студентов, которые состоят в трёх и более клубах, но за последние 90 дней не были ни на одном клубном мероприятии. Агрегация и отрицание сразу, по четырём таблицам.

```

SELECT  s.StudentID,
        s.FirstName || ' ' || s.LastName AS StudentName,
        COUNT(DISTINCT m.ClubID)          AS ClubCount
FROM    Student      s
JOIN    Membership m ON m.StudentID = s.StudentID
        AND m.MembershipStatus = 'Active'
WHERE   NOT EXISTS (
        SELECT 1
        FROM    Attendance a
        JOIN    Event      ev ON ev.EventID = a.EventID
        WHERE   a.StudentID = s.StudentID
        AND     a.RSVPStatus = 'Attended'
        AND     ev.EventDate >= CURRENT_DATE - INTERVAL '90 day'
        )
GROUP BY s.StudentID, s.FirstName, s.LastName
HAVING  COUNT(DISTINCT m.ClubID) >= 3
ORDER BY ClubCount DESC, StudentName;

```

Задание 5.1. Студенческие клубы (нотация Чена)



Обозначения: прямоугольник — сильная сущность; двойной прямоугольник — слабая сущность; ромб — связь; двойной ромб — идентифицирующая связь; овал — атрибут; двойной овал — многозначный атрибут; пунктирный овал — производный атрибут; подчёркивание — ключ; пунктирное подчёркивание — частичный ключ; одинарная линия — полное участие; двойная линия — полное участие.
